Contents

XO Arena — Performance Plan v2 **1**
Targets (binding budgets) . . . 1
Current baseline (2026-05-02 staging, cold-anon) . . . 2
Prod sanity-check baseline (2026-05-04) . . . 2
Pre-flight — before running Phase 0 . . . 3
Post-flight — clean up after the run . . . 4
Phase 0 — Re-baseline . . . 5
Section A — Real performance . . . 9
Section B — Perceived performance . . . 12
Section C — Newly heavy surfaces (deserve their own pass) . . . 13
Section C2 — Compute, cache, and architecture (added from brainstorm) . . . 14
Section D — Instrumentation as a first-class deliverable . . . 15
Section E — Aggressive bets (re-review 2026-05-04) . . . 18
Section F — Pre-launch review gaps (2026-05-04) . . . 20
Out of scope (and why) . . . 24
Sequencing *(updated 2026-05-02 from snapshot data)* . . . 25

XO Arena — Performance Plan v2

v1 lives at `doc/archive/Performance_Plan.md` and is closed out (17 phases, Ready ~330ms floor on the small surface set that existed at the time).

This v2 is a fresh pass against the *current* platform: Fly.io hosting, four services (backend / landing / tournament / packages), SSE+POST realtime, Tables-as-primitive, Multi-Skill Bots (Phase 3.8), the Intelligent Guide journey, and a mobile UI. v1's numbers and assumptions are stale; this plan re-baselines first and prescribes fixes second.

Goal: bear down on real *and* perceived performance across **every** user- facing surface — desktop and mobile, cold and warm, anonymous and signed-in, free play and tournament play.

Targets (binding budgets)

Every page on Fly prod, p75 over a 7-day RUM window:

Metric	Desktop	Mobile (4G mid-range)
FCP	≤ 100ms	≤ 250ms
Ready (spinner gone)	≤ 200ms	≤ 500ms
LCP	≤ 200ms	≤ 500ms
INP (interaction p75)	≤ 100ms	≤ 200ms
TTFB (HTML)	≤ 60ms	≤ 150ms

Backend p95 budgets (per endpoint, single-region):

Endpoint family	p95
`GET /bots`, `/leaderboard`, `/puzzles`	≤ 30ms
`GET /tournaments`, `/tournaments/:id`	≤ 100ms
`GET /users/*` (stats, history)	≤ 80ms
`POST /rt/tables` (create)	≤ 120ms
`POST /rt/tables/:slug/move`	≤ 60ms
SSE channel join → first event	≤ 80ms

Bundle budgets per route:

- Initial parse on `/`: ≤ 120 KB gz JS, ≤ 20 KB CSS
- Largest other route: ≤ +60 KB gz over baseline
- Hero image: ≤ 200 KB on mobile (~~today: 888 KB~~ → **shipped 2026-05-05 at 50 KB mobile / 174 KB desktop** via responsive

WebP, see Phase 3)

If a phase's measured win doesn't move at least one budgeted metric, mark it *landed but ineffective* and revisit the assumption.

Current baseline (2026-05-02 staging, cold-anon)

Full numbers + caveats: `doc/Performance_Snapshot_2026-05-02.md`. The single-pass median across 13 routes:

Metric	Target (desktop)	Measured	Over by
FCP	≤ 100ms	~525ms	5.3×
LCP	≤ 200ms	~640ms	3.2×
Ready	≤ 200ms	~760ms	3.8×

Metric	Target (mobile)	Measured	Over by
FCP	≤ 250ms	~1360ms	5.4×
LCP	≤ 500ms	~1450ms	2.9×
Ready	≤ 500ms	~1580ms	3.2×

Every route is over budget on both devices. Three load-bearing facts from the snapshot drive the priorities below:

1. **One bundle, every page.** First-paint JS is ~493 KB gz on every route (`vendor-react` 82 KB + `main.supported` 411 KB). The `main.supported` chunk is **1,529 KB raw / 411 KB gzip** and contains the entire app — no per-route lazy splitting at the page level today. FCP / LCP / Ready collapse to within ~100ms of each other on every route, the fingerprint of a single bundle blocking the first paint.
2. **PlayVsBot is genuinely 600–700ms slower than the next-worst route** on both devices (10-run confirmation). Not noise. The `/play?action= vs-community-bot` path runs sequential: `getCommunityBot()` → `/api/v1/rt/tables` POST → redirect, all before the spinner clears.
3. **Image bytes were not counted.** `colosseum-bg.jpg` (888 KB) didn't appear in the static byte total — likely loads async after Ready resolves. Need to confirm before promoting Phase 3.

The data does not support chasing DB indexes (Phase 2), SSE latency (Phase 5), or cross-service hops (Phase 4) yet — the bundle + parse floor is currently dominating everything else by a ~10× margin. Those phases stay in the plan but move to Tier 2.

Prod sanity-check baseline (2026-05-04)

To confirm staging is a usable proxy for prod, ran `perf-v2.js` against `xo-landing-prod.fly.dev` (`--target=prod --warmup`, 5 runs × 13 routes × desktop+mobile, cold-anon), then immediately re-ran against `xo-landing-staging.fly.dev` to remove staleness from the comparison.

Saved at:

- `perf/baselines/perf-prod-2026-05-04T10-49-09-639Z.json`
- `perf/baselines/perf-staging-2026-05-04T10-55-42-186Z.json`

Headline: prod ≈ staging within run-to-run noise. Staging is a usable proxy.

Device	Mean Δ% (prod − staging, ready p50)	Notes
Desktop	**+8.6%** (n=13)	Two routes prod-faster (Stats −8%); rest +2 to +19%
Mobile	**−0.1%** (n=13)	Spar prod-faster by 15%; everything else within ±5%

Per-route ready-p50 (ms), prod vs same-day staging:

Route	Desktop prod	Desktop stg	Δ%	Mobile prod	Mobile stg	Δ%
Home	985	878	+12%	2077	2036	+2%
Play	969	849	+14%	2051	2046	+0%
PlayVsBot	1490	1421	+5%	2061	2055	+0%
Leaderboard	968	811	+19%	2076	2053	+1%
Puzzles	850	796	+7%	2127	2051	+4%
Tournaments	906	767	+18%	2069	2059	+0%
Tables	977	883	+11%	2085	2038	+2%
Spar	860	837	+3%	1714	2012	−15%
Stats	843	918	−8%	2046	2053	−0%
Profile	913	811	+13%	2110	2052	+3%
ProfileBots	901	830	+9%	2055	2037	+1%
Gym	860	846	+2%	2059	2077	−1%
Settings	893	826	+8%	2081	2063	+1%

> **Earlier draft of this section reported +23%** because the staging comparison was a 2-day-old baseline (2026-05-02) — staging machines had been warm during that earlier sweep. A same-clock comparison shows mobile is essentially identical and desktop is within first-route cold-start variance. Lesson for the playbook: **always run both environments back-to-back** when comparing.

What this means for the plan

- **Staging is a valid proxy for prod.** Optimizations that move staging numbers will move prod numbers by approximately the same amount. Phase 1/1b acceptance can be measured on staging alone going forward; re-baseline prod once per Tier (not per phase) for confirmation.
- **The bundle + parse floor still dominates on prod.** Same JS weight (490 KB), same FCP/LCP/Ready collapse pattern. Phase 1 (per-route splitting) is still the right next move.
- **PlayVsBot tax is environment-independent.** Both prod and staging show PlayVsBot ~600ms above the rest at desktop p50. Phase 1b's sequential-init flattening applies equally to both.
- **One outlier still worth investigating:** Tournaments desktop p95 hit 6448ms on staging (single run) and 3543ms on prod's earlier run — a cold backend machine handling the first `/api/tournaments` request. `--warmup` only HEADs landing. *Fix:* extend `--warmup` to also hit `/api/version` on backend and `/api/tournaments` on tournament before measuring. File against Phase 6 (cold-start).

Follow-ups still open

- Run a one-off `Home × 5` against `aiarena.callidity.com` to confirm custom-domain TLS does not add measurable latency.
- Re-baseline prod after the next Tier-0 phase lands to confirm the staging-as-proxy assumption holds as the bundle gets smaller.

Pre-flight — before running Phase 0

Cold benchmark numbers are only meaningful if the environment is real and the data is representative. Run this checklist before every measurement pass.

Environment up

All three Fly apps + the staging DB must be reachable:

```
fly status -a xo-backend-staging \
  && fly status -a xo-landing-staging \
  && fly status -a xo-tournament-staging
curl -sf https://xo-backend-staging.fly.dev/api/version && echo OK
curl -sf https://xo-tournament-staging.fly.dev/api/health && echo OK
curl -sfI https://xo-landing-staging.fly.dev/ | head -1
```

Warm the machines (don't measure cold-start as cold-page)

Fly auto-suspends idle staging machines. The first request after suspend includes a multi-second machine boot — that's a *cold-start* number, not a cold-page number, and it'll skew every percentile. Two options:

- **Recommended for a benchmark window:** disable auto-suspend on each app for the duration of the run, then re-enable.

```
for app in xo-backend-staging xo-landing-staging xo-tournament-staging; do
  fly scale count 1 --max-per-region=1 -a "$app"  # ensure 1 machine running
  # disable auto_stop_machines in fly.toml or use the API; revert after
done
```

- **Or:** discard the first run of each route in the script (the warmup pass is built into `perf/perf.js` Phase 0.1 work).

Test data shape

Heavy pages need representative rows. Staging may be sparse. Before the run, seed (or confirm) at least:

- One in-progress 32-participant cup (for `TournamentDetailPage` worst-case).
- One user with 5+ bots, at least 2 of which have a trained ML skill (for `/profile?section=bots` and `/gym` heavy paths).
- ≥ 50 leaderboard entries with a mix of bots + humans (for `/leaderboard` disambiguation render cost).
- ≥ 1 active table (`/tables` list) and 1 spectator-joinable table.

If the data isn't there, document it in the snapshot — *don't pretend the numbers reflect prod-shape*.

Auth

`cold-signed-in` and `warm-signed-in` runs need a working test account on staging:

```
# Required env vars (same ones e2e/qa.mjs uses):
echo "$TEST_USER_EMAIL"   # must resolve
echo "$TEST_USER_PASSWORD"

# Smoke the login flow once before the benchmark:
cd e2e && BASE_URL=https://xo-landing-staging.fly.dev npx playwright test smoke --project=chromium
```

Quiesce other traffic

If anyone else is QAing on staging during the run, the numbers are noise. Coordinate a window or run after-hours.

Snapshot the build

Capture the staging git SHA so the snapshot is reproducible:

```
curl -s https://xo-backend-staging.fly.dev/api/version  # logs commit + version
```

Record it at the top of `Performance_Snapshot_<date>.md`.

Post-flight — clean up after the run

Every change made for the benchmark must be reverted, or the next person to hit staging gets a polluted environment (cost from pinned machines, polluted metrics from seed data, locked test users, etc.). Run this checklist *every* time, immediately after the snapshot is captured.

Restore Fly machine policy

If you disabled auto-suspend for the warm pass, put it back:

```
for app in xo-backend-staging xo-landing-staging xo-tournament-staging; do
  # Either revert fly.toml + redeploy, or use the API to flip
  # auto_stop_machines back to "stop". Confirm via:
  fly status -a "$app"   # machines should return to suspended on idle
done
```

If you scaled machine count up, scale back down.

Remove seeded benchmark data

Anything created *for the benchmark* must be deleted. The `um` CLI is the canonical path so cascades fire correctly (TournamentParticipant rows get cleaned up before User delete, etc.):

```
# Test cup created for TournamentDetailPage worst-case
docker compose exec -T backend node --experimental-transform-types --no-warnings \
  src/cli/um.js tournament:delete <cup-id>

# Bots minted for the heavy /profile + /gym runs
docker compose exec -T backend node --experimental-transform-types --no-warnings \
  src/cli/um.js bot:delete <botId>   # repeat per seeded bot

# Throwaway users (anything tagged isTestUser:true that isn't the persistent
# QA account) — confirm before bulk delete
```

If a Phase 0 helper script seeded the data, it should ship with a paired `--cleanup` flag so the teardown is one command, not a manual list.

Reset RUM ingest

Phase 0.2 wires `web-vitals` → `POST /api/v1/rum`. The benchmark's synthetic runs will dump hundreds of fake samples into `RumSample` and skew the next day's percentiles. Either:

- Tag every benchmark sample with `source: 'synthetic-baseline'` and exclude it from RUM dashboards by default; or
- Delete the rows the run produced:

  ```
  DELETE FROM "RumSample"
  WHERE "createdAt" >= '<run-start-ts>'
    AND "createdAt" <= '<run-end-ts>';
  ```

Same logic for backend RED metrics if they're persisted (Prometheus scrapes are ephemeral, so usually nothing to do there).

Restore CI / config drift

If you flipped any flag or env var on staging for the run (e.g. enabled extra logging, raised a cache TTL to test something, turned off a feature gate), revert before walking away. List them in the snapshot doc so nothing's forgotten.

Sanity check

One last pass to confirm staging is back to its normal shape:

```
fly status -a xo-backend-staging              # auto_stop policy normal
curl -s https://xo-backend-staging.fly.dev/api/version | grep -i version
docker compose exec -T backend node --experimental-transform-types --no-warnings \
  src/cli/um.js list --filter=isTestUser     # only persistent QA rows remain
```

Smoke the e2e suite once to confirm nothing regressed during cleanup:

```
cd e2e && BASE_URL=https://xo-landing-staging.fly.dev npx playwright test smoke --project=chromium
```

If anything fails, the snapshot is suspect — re-run after fixing.

Phase 0 — Re-baseline

v1's `perf/perf.js` ran 6 pages on Railway with 2 services. We now have ~20 routable surfaces on Fly with 4 services and a totally different realtime stack.

0.1 Synthetic baseline (Playwright) *(done — 2026-05-02)*

Shipped in `perf/perf-v2.js`. First baseline captured against staging on 2026-05-02 — 130 measurements (13 routes × 2 device profiles × 5 runs) written to `perf/baselines/perf-staging-2026-05-02T*.json`. Analysis in `doc/Performance_Snapshot_2026-05-02.md`.

- ☒ Extend `perf/perf.js` to cover the full route inventory.
- ☒ Two device profiles: desktop (1280×800, broadband) + mobile (Moto G4 / 4G via CDP throttling).

- ☒ Run against `xo-*-staging.fly.dev`. 5 runs each, p50 + p95.
- ☒ Capture: TTFB, FCP, LCP, Ready, JS bytes, image bytes, requests.
- ☒ Persist results into `perf/baselines/<env>-<ts>.json`.
- ☐ **Open follow-ups** — these are real gaps, *not* blockers for Phase 1; tackle when their inputs become available:
  - ☐ Wire `cold-signed-in` and `warm-signed-in` contexts (needs `TEST_USER_EMAIL` + `TEST_USER_PASSWORD` in env).
  - ☐ Run against `xo-*-prod.fly.dev` once Phase 3.8 is promoted.
  - ☐ Re-run on staging once it bumps past `1.3.0-alpha-8.0` so Sprint 3.8.B + 3.8.C are reflected.
  - ☐ Add INP measurement (one scripted interaction per page — open modal / click row / submit form).
  - ☐ Disable Fly auto-suspend on staging for the run, seed the four representative-data fixtures from the Pre-flight section.

0.2 RUM (real-user monitoring) *(not started — Tier 1)*

- ☐ Wire `web-vitals` into the landing app — POST `{ metric, value, route, deviceClass }` to a new `POST /api/v1/rum` endpoint with sampling (10% in prod, 100% in staging).
- ☐ Server stores into `RumSample` table (route, metric, value, percentile bucket, timestamp, country/region). Add Prisma model + migration.
- ☐ Admin dashboard panel: per-route p50/p75/p95 over 24h / 7d. (Reuse the existing admin metrics scaffold.)
- ☐ Define alert: any route's p75 Ready > 1.5× target for 30min → Slack.

0.3 Backend RED metrics (Rate / Errors / Duration) *(not started — Tier 2)*

- ☐ Add request-duration histogram middleware in `backend/src/index.js` and `tournament/src/index.js`. Bucket per route + status.
- ☐ Expose `/metrics` (Prometheus text format) — Fly already supports scraping.
- ☐ Backfill the Observability dashboard (`doc/Observability_Plan.md`) with p95 panels per endpoint family.
- ☐ Confirm SSE channel join + dispatch latency are instrumented (the realtime postmortem mentions per-channel timing — verify it lights up Grafana).

0.4 Targeted gap measurements (2026-05-04)

The 0.1 baseline left four open questions. New scripts plug each one without waiting on RUM (0.2) or Prometheus (0.3):

- **`perf/perf-inp.js`** — INP per page via `PerformanceObserver({type: 'event', durationThreshold: 16})`, 5 runs × desktop+mobile.
- **`perf/perf-sse-rtt.js`** — SSE connect, POST move ack, POST move → SSE state event, POST move → bot move event, 20 runs.
- **`perf/perf-backend-p95.js`** — concurrent loader, 200 reqs × concurrency 5 across the five hot read endpoints.
- **`perf-v2.js --extended-resources`** — wait 5s post-Ready and re-collect resource bytes, exposing late-loading images that `transferSize` reports as 0 at Ready time.

Saved at `perf/baselines/{inp,sse-rtt,backend-p95,perf}-{env}-*.json`. Headlines below; raw numbers in the JSON.

INP — the click-to-paint floor is excellent Where the script could find a stable interaction, every measured route sits **well under the 200ms "good" threshold** on both desktop and Moto G4 mobile:

Interaction	Desktop p50	Desktop p95	Mobile p50	Mobile p95
Home — refresh demo	**24ms**	120ms	24ms	32ms
Home — open sign-in	**32ms**	32ms	24ms	40ms
Puzzles — first button	**24ms**	24ms	24ms	24ms

Two routes (Tournaments filter, Leaderboard "Show bots" toggle) hit selector-miss issues during the run; the data we have already says **INP is not a problem on the platform today** for the routes that landed. Phase 15 (Guide INP audit) drops in priority — the *idle* floor is fine. The remaining concern is INP under live SSE updates, which the current script doesn't measure (TODO: extend with a "during-cup" mode).

SSE round-trip — the real perceived-latency story Every PvP / PvB move passes through this path. Numbers are the headline finding of this whole section:

Phase	Staging p50	Staging p95	Prod p50	Prod p95
SSE connect → session	194ms	258ms	179ms	263ms
POST move → ack	173ms	233ms	185ms	252ms
POST move → SSE state	**656ms**	902ms	**577ms**	926ms
POST move → bot move event	657ms	902ms	578ms	926ms

(2026-05-05 baselines — staging `sse-rtt-staging-...01-10-39-968Z`, prod post Fly-Replay fix `sse-rtt-prod-...01-30-53-025Z`. 20/20 successful runs on both envs, no failures.)

Three load-bearing facts here:

1. **The POST acks at 174–262ms but the SSE event arrives 386–408ms later.** That gap is the SSE pub/sub dispatch — flushing the channel write through to the same client's open EventSource. Players click a square and wait ~half a second to see the result. This is *the* perceived-perf bottleneck on every move, every game, every user. **Phase 5 promotes from Tier 2 to Tier 0.**
2. **Bot move and player move arrive ~simultaneously** because the backend dispatches both state events in the same request handler. Bot computation is effectively free. No work needed there.
3. **Prod is ~120ms slower than staging on the SSE round-trip.** Same machine class; same code. Likely cold-machine flap on the move POST handler — Phase 6 input.

Multi-machine Fly-Replay fix — 2026-05-04 The prod baselines collected on 2026-05-04 surfaced a real production bug, not a measurement artifact. The first prod run after the iad migration showed **18/20 SSE round-trips failing** with `409 SSE_SESSION_EXPIRED`; staging was 0/20.

Root cause: prod runs **2 backend machines** behind Fly's round-robin load balancer. The SSE session registry is a per-process `Map` keyed by session id, so any `/rt/*` POST that hits the *other* machine looks up an unknown session and 409s. Staging (1 machine) never tripped it.

Fix (commit `0771718`): SSE session ids are now minted with a machine-id prefix (`<FLY_MACHINE_ID>.<nanoid>`). The `/rt/*` middleware decodes the prefix and, if it doesn't match the current machine, returns the `Fly-Replay` header to retry the request on the owning machine. ~30 LOC primitive in `backend/src/realtime/flyReplay.js` plus 2 call sites.

Post-fix prod baseline: **0/20 failures, 20/20 valid samples.** Code- path overhead is unchanged (`server.lookup` 4ms, `server.apply` 8ms).

Baseline discontinuity to know about: `perf/baselines/sse-rtt-prod-2026-05-04T23-53-58-761Z.json` and earlier prod F4 baselines were computed over the 2 lucky runs that landed on the SSE-owning machine, so their p50/p95s look artificially low. Use `perf/baselines/sse-rtt-prod-2026-05-05T01-17-08-425Z.json` (the first post-fix run) as the new prod F4 reference. Staging baselines are unaffected — staging never had the bug.

A future Redis-backed session registry (which would replace Fly-Replay entirely and enable non-Fly hosting) is captured in `doc/Future_Ideas.md` and is deferred until non-Fly hosting is on the table.

Prod re-baseline — 2026-05-05 (full suite, post Fly-Replay fix) `perf/perf-rebaseline.sh prod` ran all 7 scripts against v1.4.0-alpha-4.0 in 555 s. **Headline takeaways:**

- **Backend latency, all green.** No endpoint p95 over 140ms; only flag is the Better Auth rate limit on synthetic `get-session`.
- **SSE 0/20 failures.** The Fly-Replay fix is doing its job in prod. Multi-machine routing no longer drops POSTs.
- **Cold-page Ready** — desktop p50 ~750–990 ms across 13 routes; mobile (Moto G4 / 4G) p50 stays in a tight ~2030–2080 ms band. Mobile is dominated by JS parse/eval (496 KB bundle) + the 888 KB hero image — the band is so flat *because* every route ships the same payload.
- **TBT (long tasks)** — desktop = 0 ms across the board. Mobile Home p50 = **99 ms** (p95 121 ms). All TBT lives in mobile JS parse, not application code.
- **INP** — every measured interaction p50 ≤ 32 ms (desktop and mobile). The "click → paint" floor is a non-issue. (Rankings toggle still produces 0 samples because the toggle settles below the 16 ms PerformanceObserver threshold — re-anchor that probe to a heavier interaction in a future pass.)

The two genuinely load-bearing items the new baselines re-confirm:

1. **888 KB hero image on every cold-anon page** — the largest single-byte cost after the JS bundle, and the only one that's imperceptibly degradable (because of the 6–18% photo opacity overlay). See "Hero candidates" table below.
2. **POST move → SSE state ~577 ms p50 prod** — every move waits roughly half a second to see the result. Decomposition shows `publishToPickup` (Fly Upstash pub/sub) is the long pole at 383 ms p50 / 639 ms p95.

Everything else is in the *no longer the bottleneck* bucket.

Backend endpoint p95 — healthy across the board

Endpoint	Stage p50	Stage p95	Stage p99	Prod p50	Prod p95	Prod p99
`GET /api/version`	52ms	135ms	225ms	54ms	139ms	158ms
`GET /api/v1/bots?gameId=xo`	72ms	184ms	305ms	68ms	140ms	157ms
`GET /api/v1/leaderboard?game=xo`	52ms	113ms	137ms	53ms	120ms	144ms
`GET /api/auth/get-session`	57ms	139ms	147ms	57ms	135ms	206ms
`GET /api/tournaments`	67ms	143ms	256ms	54ms	137ms	160ms

(2026-05-05 baselines. All endpoints 200/200 ok except prod `get-session` at 90/200 — Better Auth's default rate limiter is still hitting the synthetic harness at concurrency 5. Real users won't trip it; whitelist or raise the cap for the synthetic runner.)

Implications:

- **Phase 2 (DB index pass) is genuinely off the critical path for now.** No endpoint's p95 is over 175ms; p99 caps at 242ms (cold-bot list on prod). Promote only when a future feature plants a slow query.
- **Prod auth endpoint rate-limited at concurrency 5.** 110 of 200 requests returned 429 — Better Auth's default rate limiter is kicking in hard. Either bump the limit or whitelist the smoke runner. Either way: real users won't hit this, but admin scripts will.
- **`/api/v1/bots` is the slowest endpoint** (133ms p50 prod). Every cold-anon page that calls `getCommunityBot()` waits on this. If Phase 1 cuts page-level work, this becomes the next visible blocker — candidate for edge KV (Section E4) since the built-in bot list rarely changes.

Extended-resource capture — the hero image was never zero KB The 2026-05-02 snapshot reported `img_kb: 0` on every route. With `--extended-resources` (5s post-Ready wait), `colosseum-bg.jpg` shows up at **909 KB** on every measured route, starting around 500–700ms (overlapping the Ready window):

Route	Image start (ms)	Bytes
Home	534	909 KB
Leaderboard	676	909 KB
Tournaments	507	909 KB

Original perf-v2 was polling `transferSize` while the resource was still in flight, which Resource Timing reports as 0 until the body fully lands. **Phase 3 (image diet) promotes from Tier 1 to Tier 0**: ~888 KB shipped on every page is the second-biggest single-byte cost after the JS bundle, and unlike JS it's reachable with one PR (WebP + responsive sizes + heavy compression that the 6–18% photo opacity makes invisible).

Hero candidates evaluated 2026-05-05 (in `perf/hero-candidates/`):

Candidate	Spec	Size	vs original
Original	1920×1279 JPEG	888KB	—
A	1600w WebP q70	174KB	**−80%**
B	1280w WebP q55 (browser scales)	94KB	**−89%**
C	960w WebP q60 + sharpness 7 blur	64KB	**−93%**
D	800w WebP q50 + sharpness 7 blur	47KB	**−95%**

Because the image renders at `--photo-opacity: 0.18` (light) / `0.06` (dark), encode-time blur and aggressive downscale are imperceptible — the eye sees a tinted wash, not a photograph. Recommendation was: ship **B** as the default with WebP, keep the JPG as a fallback for the ~3% browsers without WebP support.

Shipped 2026-05-05 (commits `28b7aca` + `f14d052` /stage to staging v1.4.0-alpha-4.1) — went with a 2-asset responsive setup via CSS `@media (min-width: 768px)` instead of a single B candidate, so 4G mobile gets the smaller 800w/q55+blur asset (D) and tablet+ gets the 1600w/q70 asset (A):

Tier	File	Size	Spec
Mobile	`colosseum-mobile.webp`	50KB	800w q55 sharpness 7
Tablet+	`colosseum-desktop.webp`	174KB	1600w q70

Original `colosseum-bg.jpg` (888 KB) kept in `/landing/public/` as a silent fallback for any future need.

Measured impact (staging v1.4.0-alpha-4.1, 2026-05-05 baseline):

Metric (Home, cold-anon)	Before	After (staging)	Delta
`img_kb` mobile	888 KB	**50 KB**	**−94%**
`img_kb` desktop	888 KB	**174 KB**	**−80%**
Mobile TBT p50	62 ms	50 ms	−19%
Mobile LCP p50	1816 ms	1792 ms	−1%
Mobile Ready p50	2052 ms	2041 ms	−1%

The Ready/LCP movement on mobile is small because mobile cold-anon is JS-parse-bound on Moto G4 (Phase 1 territory); the WebP win lands primarily on **bytes-over-the-wire** (~838 KB saved per cold mobile visit on 4G) and **TBT** (less main-thread image-decode work), not on synthetic Ready. Real users on metered networks feel the byte savings more than the synthetic harness does.

This **closes Phase 3** as Tier 0. The next Tier 0 item is Phase 1 (JS bundle splitting).

Deliverable: a single `Performance_Snapshot_<date>.md` checked in next to this plan, showing where every route currently sits vs the targets above. *Every subsequent phase compares against this baseline.*

Section A — Real performance

Phases that move the actual latency floor (server time, network, parse, render). Ordered by expected impact-to-effort.

Phase 1 — Bundle audit + per-route splitting *(Tier 0 — start here)*

The 2026-05-02 visualizer pass (run `VISUALIZE=1 npx vite build` in `landing/`) showed `main.supported-*.js` at **1,529 KB raw / 411 KB gz** holding the entire app. Only `vendor-react`, `game-xo`, `game-pong`, and the Gym sub-tabs are split out — every page component is in `main`. Per- route splitting is the highest-leverage fix in the entire plan; until this lands, every other phase is a rounding error.

Concrete sub-steps (in order):

- ☐ **Per-route `React.lazy()` for every page in `App.jsx`.** PlayPage, GymPage, TournamentDetailPage, TournamentsPage, ProfilePage, BotProfilePage, PublicProfilePage, RankingsPage, StatsPage, SettingsPage, TablesPage, TableDetailPage, SparPage, PuzzlePage, plus all `/admin/*` pages. Wrap each `<Suspense fallback={<Skeleton/>}>` so the skeleton already aligns with Phase 8 work.
- ☐ **Vendor-split `@xo-arena/*`** in `manualChunks` — `xo`, `nav`, `ai`, `sdk`. They're currently bundled into `main.supported`. Promote each to its own chunk.
- ☐ **Audit Better Auth client.** Likely a meaningful chunk (auth-client + better-fetch + zod). If the auth flow is only relevant after a sign-in trigger, lazy-load the client module.
- ☐ **Audit `recharts`.** Only used in Gym Analytics + Stats. Confirm `vendor-charts` chunk separation; if it's leaking into `main`, fix the import boundary.
- ☐ **Audit ML engines (`@xo-arena/ai`).** Q-learning / DQN / AlphaZero should only load on Gym Train tab — already lazy at the tab level, but verify the engines aren't statically imported elsewhere.
- ☐ **Audit images / SVG / icons.** Anything imported as a JS module should be inspected; large inline SVGs should move to `<img>` or a sprite sheet (currently inline JSX SVGs ship inside the JS chunk).
- ☐ **Tactical library swaps.** Each of these has a measured-bytes payoff; do them as separate PRs so each can be validated:
  - `recharts` → `uplot` (~20× smaller, only used in Gym Analytics + Stats).
  - Audit `dayjs` boundary; nothing should pull `moment`, `luxon`, etc.
  - Subset `lodash` imports (`lodash-es/get`, not `lodash`); remove `lodash` if any path still uses it.
  - Audit `zod` boundary — only use on validators that actually run client-side.
- ☐ **Tree-shake hygiene.** Mark `sideEffects: false` on every internal `@xo-arena/*` package so unused exports drop. Audit duplicate deps (`npm ls react`, `npm ls zustand`).
- ☐ **Brotli at the edge.** Confirm Fly is serving Brotli, not just gzip — usually 15–20% smaller wire bytes for free.
- ☐ **Aggressive minifier.** Compare `esbuild` vs `swc` vs `terser` passes on `main.supported`. Cumulative ~5–10% gain typical.
- ☐ *(Optional / longshot)* Preact compat alias — saves ~50 KB gz over React 18, but Better Auth + react-router compat needs verification. Land last, behind a feature flag.
- ☐ Generate updated visualizer report; commit at `perf/baselines/bundle-<date>.html` for diffing.
- ☐ Re-run `perf-v2.js` against staging — confirm `main` < 200 KB gz, per-route chunks 30–80 KB gz, and FCP / Ready drop on every page.

- ☐ **CI bundle-size guard.** Fail PR if any chunk grows > 5% over `main` without a `perf-ok` label. (`size-limit` or hand-rolled.)

Expected outcome (data-driven hypothesis): - `main` chunk: 411 KB gz → ≤ 200 KB gz - First-paint JS: 493 KB gz → ~280 KB gz on Home, lower per-route - Mobile FCP: 1360ms → ~700ms (parse cost cut roughly proportional to byte cut on Moto G4 CPU) - Desktop Ready: 760ms → ~400ms

If the second perf-v2 run does *not* show these moves, Phase 1 is *landed but ineffective* and we go back to the visualizer to find what else is stuck in `main`.

Phase 1d — Instant-ready hero (inline SVG + late hydrate) ***(Tier 0, perceived-only)***

A pure perceived-perf trick that pairs with Phase 1: render a static SVG of a tic-tac-toe board (mid-game position) inlined in `landing/index.html`. It paints the moment the HTML lands. When the JS bundle finishes parsing, `DemoArena` hydrates and the live game takes over (cross-fade or in-place swap). The user never sees a blank hero or spinner — first paint *is* the hero.

What it actually buys (relative to current 2026-05-05 baseline, on top of whatever Phase 1 lands):

Segment	FCP today	After Phase 1	After Phase 1 + 1d
Mobile cold-anon (Moto G4 / 4G)	1416ms	~700ms	**~200ms**
Desktop cold-anon	676ms	~400ms	**~50ms**
Mobile repeat (cache hit)	600ms	~300ms	**~150ms**
Desktop repeat (cache hit)	150ms	~150ms	**~50ms**

LCP follows the same delta because the SVG *is* the largest contentful element. The Ready metric is unchanged — bundle parse still happens — but the perceived "the page is alive" moment slides to the HTML-render boundary.

Why mobile and desktop both win:

- Mobile: −1200ms is the headline; pushes the "page feel" from "loading" to "instant" (crosses the well-known ~200ms perception threshold).
- Desktop: −580ms is real even on a fast connection; takes a "loaded fast" feel into "feels native". First-impression / conversion cohort benefits.

Mechanics:

- ☐ Author a static SVG of an X-O board mid-game (~1-2 KB markup), inlined in `landing/index.html` inside the hero container. Style with the same Tailwind classes as `DemoArena`'s React render so the visual is identical.
- ☐ Lock the box dimensions explicitly with CSS `aspect-ratio` and `width`/`height` so the SVG → React-render swap doesn't shift layout (CLS budget ≤ 0.1).
- ☐ On `DemoArena` mount, fade the static SVG out and the live board in over ~200ms (or snap if the styles are pixel-identical and there's no flash).
- ☐ Wire any in-hero buttons (`Sign in`, `Watch another match`) so a click before JS-ready queues the action and React Router picks it up on hydrate. No "loading" flash on click.

Effort: ~1 day. **Real ms saved:** 0 (page is no faster, just appears faster). **Perceived ms saved:** −1200ms FCP/LCP mobile, −580ms desktop — bigger than any other single phase. **Stacks with Phase 1** — independent code paths, ships in parallel.

Risks:

1. *Visual drift between SVG and React render.* Mitigation: generate the SVG from the same Tailwind classes / a shared React component used in both modes, or freeze the styles with a snapshot test.
2. *CLS on swap.* Mitigation: explicit box dimensions in CSS, verify in measurement gap #7 (CLS).
3. *Click-before-hydrate.* Mitigation: native anchor links for navigation buttons; queue-and-replay pattern for the rest.

Phase 1b — PlayVsBot deep-dive ***(Tier 0, immediate after Phase 1)***

The 10-run confirmation showed `/play?action=vs-community-bot` is 600–700ms slower than the next-worst route on both devices, *every time*. The path is sequential and visible to a returning user every time they click "Play vs Bot" cold:

```
mount → getCommunityBot() fetch → /api/v1/rt/tables POST → redirect → render
```

- ☐ Trace each step server-side and client-side; identify the longest synchronous wait.
- ☐ Render an interim board *shell* (skeleton board + "Finding bot…") *before* `getCommunityBot()` returns — the spinner detection then flips when the bot is ready, not 600ms later.
- ☐ Move `getCommunityBot()` cache warming into the route entry — hover-prefetch `/play` from the home CTA, fire the bot fetch

then.

- ☐ Verify `/api/v1/rt/tables` POST is the actual path (not socket). Make sure the server returns the table id immediately and SSE backfill picks up state, instead of a single big response.
- ☐ Re-measure with the same `--routes=PlayVsBot --runs=10` invocation. Target: parity with `/play` (≤ 750ms desktop, ≤ 1600ms mobile).

Phase 2 — Database query audit + indexing pass

The schema has grown a lot since v1. Audit slow queries across hot endpoints, especially the new ones:

- ☐ Enable Postgres `pg_stat_statements`; collect 24h of staging traffic.
- ☐ Top 20 by total time + top 20 by mean time — review each.
- ☐ Critical paths to check: tournament list (`/api/tournaments`), tournament detail with rounds + participants + matches eager-loaded, leaderboard, bot list with skills, ML session list, table presence lookups.
- ☐ Add missing indexes (most likely: `TournamentMatch (tournamentId, roundId)`, `TournamentParticipant (tournamentId, status)`, `BotSkill (botId, gameId)` — verify the partial unique still covers reads).
- ☐ Convert any lingering N+1s to `findMany` + grouped indexing (`groupBy`/`include`).
- ☐ **Materialized view for the leaderboard.** The current SQL aggregates win counts on every request. Refresh the view on game-end (already a single hot path). Read becomes O(rows returned), write cost moves out of the request thread.
- ☐ **Read replica for hot reads** (Fly Postgres supports it). Public read endpoints (`/leaderboard`, `/bots`, `/tournaments`) go to the replica; writes + auth stay on primary. Cuts write- lock contention on busy windows.
- ☐ **Stream large responses.** `GET /api/tournaments/:id` currently returns the full eager tree as one payload. With Express `res.write()` chunks (or NDJSON), the client can paint the header → bracket → participant table → match history in stages instead of waiting on the longest one.
- ☐ **Worker queue for off-request work.** ML training kicks, ELO calibration, journey event publishing — every one of these blocks the request that triggered it today. Push to BullMQ-style queue (Redis-backed); the request returns immediately.
- ☐ **Pre-warm critical caches on boot, not first request.** Built-in bot roster, puzzles, system config — load on `app.listen` callback so the first user doesn't pay the miss.
- ☐ **Compress payload responses.** Confirm gzip/Brotli on JSON responses, not just static assets — cuts wire bytes 60–80% for tournament/leaderboard JSON.

Phase 3 — Hero image + asset diet *(Tier 1 — confirm impact first)*

The 2026-05-02 baseline showed **0 KB image bytes** on every route — the `colosseum-bg.jpg` (888 KB) didn't appear in the static byte total. It likely loads async after Ready resolves, or via a CSS background not caught by the resource-type filter. Before sinking time into AVIF encoding, confirm where and when the image actually lands.

- ☐ **Investigate first.** Open DevTools → Network on `/` cold, filter by Img. Confirm whether `colosseum-bg.jpg` loads at all on cold-anon, when it loads relative to FCP / Ready, and on which routes.
- ☐ If it loads after Ready: low-priority — it doesn't move the measured budget. Still worth shipping AVIF + responsive sizes for mobile data costs, but demote out of Tier 1.
- ☐ If it blocks LCP on `/`: keep in Tier 1 — convert to AVIF + WebP + JPEG fallback via `<picture>`; cap mobile delivery at ≤ 1280 wide; preload.
- ☐ Audit other public images (`landing/public/`); drop anything unused.
- ☐ Add `loading="lazy"` and explicit `width`/`height` on every `<img>` to stop layout shift (CLS).
- ☐ Confirm font strategy: `font-display: swap` and self-hosted (no blocking Google fonts).
- ☐ **Subset fonts** to the glyphs actually used. A full Latin Inter subset is ~25 KB; full font is ~120 KB.
- ☐ **SVG sprite sheet** for icons. Inline JSX `<svg>` ships inside the JS chunk — moving icons to an external sprite (`<use href />`) lets them stream + cache separately and shrinks `main.supported`.
- ☐ **Skip the hero on mobile entirely.** A CSS gradient or solid colour as fallback; serve the real image only at ≥ 1024 viewport. Cuts ~200 KB off mobile data on `/`.

Phase 4 — Cross-service network shape

The landing server proxies `/api/(classification|recurring|tournaments)/*` to the tournament service. Each proxied call adds a hop.

- ☐ Measure: cross-service p50/p95 vs same-service.
- ☐ If the gap is meaningful, consider:
  - Direct browser → tournament service via a `tournament.<env>.fly.dev` subdomain + CORS (saves the proxy hop).
  - Or, push read-mostly endpoints (recurring list) into the backend with Redis-backed sharing.
- ☐ Audit `Connection: keep-alive` between landing↔backend, landing↔tournament — don't pay TLS handshake on every request.

- ☐ Confirm **HTTP/3 (QUIC)** is enabled on all Fly fronts. Cuts handshake latency on flaky mobile connections (vs HTTP/2's head-of-line blocking).
- ☐ **Same-origin auth cookie path.** Better Auth sometimes triggers a CORS preflight `OPTIONS` for `/api/auth/get-session`. If the cookie domain is set tight, this is `Access-Control-Max-Age` away from a noticeable savings (or eliminate the preflight entirely).
- ☐ **Audit cookie size.** Better Auth tokens can balloon (esp. with provider claims). Every request carries them — keep < 4 KB.
- ☐ **Combine cold-start API calls into `/api/v1/init`.** A returning signed-in user fires session check + bots + tournaments + journey progress in parallel; one combined endpoint lets the server fan out + return one payload, cutting N waterfalls to 1.

Phase 5 — SSE channel + POST round-trip latency

The realtime layer was rewritten between v1 and v2 (see `doc/Realtime_Migration_Postmortem.md`). It needs its own performance lens:

- ☐ Measure: `EventSource` open → first event latency; channel join → server ack; client POST move → server ack → SSE state event arrival.
- ☐ Look for redundant subscriptions in `useEventStream` (one channel per tab vs one shared singleton — the postmortem mentioned a singleton fix; verify it's still in place).
- ☐ Move idle/heartbeat traffic to a long-poll or coalesce into one channel to keep open connections low (Fly per-instance HTTP limits).
- ☐ Backpressure: under load, what happens when one slow consumer holds an SSE write? Add a watchdog.

Phase 6 — Backend cold start / warm path

Fly machines suspend on idle. First-request latency after suspend can dominate Ready for low-traffic regions.

- ☐ Measure cold-start time per service (backend, tournament, landing).
- ☐ Decide per service: `auto_stop_machines = false` for hot services (backend), keep for cold (admin?). Document the cost trade-off.
- ☐ Pre-warm critical caches (built-in bots, puzzles) on boot, not on first request.
- ☐ Move synchronous init out of the request path (Prisma engine load, journey config load).

Phase 7 — Mobile-specific

Mobile is now first-class but never benchmarked. The 2026-05-02 snapshot showed Mobile FCP at ~1360ms — that's the parse cost on a Moto G4 CPU. After Phase 1 cuts the bundle, the remaining mobile gap will come from critical-path render and CSS.

- ☐ Identify any desktop-only assumption: large tables, hover-only UI, keyboard shortcuts, missed touch targets.
- ☐ Audit DOM size on the heaviest mobile pages (TournamentDetailPage, GymPage with tabs).
- ☐ Lighthouse mobile score ≥ 90 per route as a hard CI gate.
- ☐ **Critical CSS inlined into `<head>`.** Most CSS is render-blocking today (`<link>` tag → server round-trip on bad networks). Extract the above-the-fold subset and inline it; defer the rest. Likely saves 50–150ms on Moto G4 / 4G FCP.
- ☐ **`touch-action: manipulation`** on tappable areas. Eliminates any residual 300ms tap delay; cheap one-liner.
- ☐ **Transform-only animations.** Audit any animation that uses `top`/`left`/`width`/`height` — replace with `transform` so it runs on the compositor thread, not main.
- ☐ **`content-visibility: auto`** on long off-screen sections (the tournament participant table when collapsed, the leaderboard rows below the fold, the Gym session list). Free skip-render for non-visible content; one CSS line per container.
- ☐ **Reduce DOM nodes** on mobile-heavy pages. TournamentDetailPage's 1900 lines render hundreds of nodes — split via Suspense and only mount visible sections.
- ☐ Service Worker for offline-first shell (covered in Phase 21 below).

Section B — Perceived performance

These don't move the latency floor but make the platform *feel* faster. v1 covered skeletons, hover prefetch, optimistic moves, optimistic writes — all good and shipped. v2 extends to the new surfaces.

Phase 8 — Skeletons everywhere new

Every page added since v1 should have a content-shaped skeleton, not a spinner.

- ☐ `TournamentsPage` — list of tournament cards skeletons.
- ☐ `TournamentDetailPage` — header + bracket + participant table skeletons.
- ☐ `BotProfilePage` — bot header + skill cards + ELO chart.
- ☐ `Profile` — accordion frames render with skeletons inside.

- ☐ `Spar` / `Tables` — table card skeletons.
- ☐ Audit: any place where a `<Spinner />` is the first thing rendered after route change is a skeleton candidate.

Phase 9 — Optimistic / streaming everywhere

- ☐ Tournament register: optimistic "Registered □" pill before the server ack (with rollback on NO_SKILL etc.).
- ☐ Bot create: bot card appears immediately with a faded "saving…" badge.
- ☐ Skill add: pill appears in `setBots` optimistic merge (already done in Profile — verify Gym uses the same path).
- ☐ Move events: already optimistic for player; verify bot moves animate smoothly when the SSE event arrives.
- ☐ Streaming responses for large GETs (tournament detail with 100+ matches) — render the bracket as JSON streams in.
- ☐ **Partial-UI loading states.** Replace `Loading…` with progress narration: "Found bot, building table…" / "Bracket ready, fetching participants…". Same wait time, lower perceived latency.
- ☐ **Animation cover during fetch.** A 200ms fade/slide on route change can hide a 200ms fetch entirely — the wait reads as polish.

Phase 10 — Hover-intent prefetch + "warm next" patterns

v1 added hover prefetch on nav links. Extend:

- ☐ On `/tournaments`, hover-prefetch each tournament's detail JSON.
- ☐ On `/profile?section=bots`, hover-prefetch `/gym?bot=:id` chunks for each row.
- ☐ On match end (post-cup), prefetch the next likely route (next match, coaching card).
- ☐ On `/play`, after first move, prefetch the rematch flow's chunks.
- ☐ **Pre-render the next likely route** in a hidden iframe / shadow DOM during idle time on slow paths (post-cup celebrations, match results). Click feels instant.

Phase 11 — Route transition polish

- ☐ Re-evaluate the v1 fade transition under React 18 Suspense — does it conflict with streaming UI?
- ☐ Add `view-transitions` API where supported (Chrome) for snap-zero route transitions on the same-document navigations.

Phase 12 — In-game feel

- ☐ Confirm sound latency is still nailed (the capture-phase pointerdown listener in `soundStore.js` — per CLAUDE.md, must not be removed).
- ☐ Animation budgets: every move, popup, modal — none > 200ms.
- ☐ Reduce-motion respect on every animation.

Section C — Newly heavy surfaces (deserve their own pass)

Phase 13 — Tournament page perf

`TournamentDetailPage` is the heaviest page in the app (1900+ lines, eager loads rounds + participants + matches + their users). It runs before / during / after the cup journey and shows under load on Cup days.

- ☐ Hard target: cold mobile Ready ≤ 600ms even with 32 participants and a full bracket.
- ☐ Split the page: bracket + participant table + match history are independent, render each behind its own Suspense boundary.
- ☐ Server: paginate completed matches; only the current round needs full detail.
- ☐ Memoize bracket layout math; profile re-renders when SSE events fire.
- ☐ Confirm `ParticipantTable` disambiguation (just shipped in 3.8.C) doesn't do per-render allocation.
- ☐ **`useDeferredValue`** on the bracket + participant lists when SSE pushes a new event. Render the new state at lower priority than user input (React 18).
- ☐ **Memoize `BracketMatch`** — currently re-renders on every SSE event for the whole bracket, even matches that didn't change.
- ☐ **`content-visibility: auto`** on rounds the user has scrolled past or off-screen.

Phase 14 — Gym page perf + ML compute

The Gym detail panel re-mounts whole tab trees on bot/skill switch, and ML inference today runs on the main thread (jank during play).

- ☐ Ensure keep-alive (`display: none`) survives the new bot→skill drilldown.
- ☐ ML model fetch should stream — show training history first, then current model weights.
- ☐ **Web Worker for ML inference.** Offload `getMoveForModel` for ML bots to a worker so the main thread never janks during a move.

- ☐ **WASM minimax / AlphaZero.** Could move bot move latency from ~30ms to <5ms. Compile the minimax engine to WASM behind a flag; benchmark vs JS. Bigger payoff once Connect4 lands (state space grows).
- ☐ **OffscreenCanvas for game render** (where applicable). Frees main thread for UI during high-FPS games like Pong.
- ☐ **`requestIdleCallback`** for journey events, telemetry, and RUM flushes — never compete with user input.

Phase 15 — Intelligent Guide journey

The journey adds bus events, popups, scrim spotlights, and re-renders on every step trigger. Measure — it's a likely INP regressor.

- ☐ INP audit on each step trigger; confirm < 100ms.
- ☐ Memoize `JourneyCard` re-renders.
- ☐ Lazy-load step-specific components (TrainGuidedModal, BotCreatedPopup).
- ☐ Confirm guide store mutations don't cascade through the whole tree (split selectors).

Phase 16 — Tables-as-primitive overhead

Tables underlie spar, journey demos, PvB, PvP, and tournament HvB. Verify the common path is hot:

- ☐ `tablePresence` lookups: O(1) per join? Memory footprint at 1000 active tables?
- ☐ `botGameRunner`: parallel runs OK at 50 concurrent demo tables?
- ☐ `createTableTracked` resource counter overhead — single Redis call or multi-hop?

Section C2 — Compute, cache, and architecture (added from brainstorm)

These are bigger swings than the per-phase tactics above. Each one is a "could be a project" rather than a checklist item, but they're listed here so the option is visible when the data points to them.

Phase 19 — Compute relocation

Today, all heavy compute runs on the user's main thread:

- ML training (Q-learning, DQN, AlphaZero) — browser-only.
- ML inference (`getMoveForModel`) — main thread.
- Game logic (minimax, AlphaZero MCTS) — main thread JS.
- Journey events, ELO calibration — request thread on backend.

Targets after this phase:

- ☐ **Web Worker for ML inference.** Offload to a worker; main thread never janks during a move. Pairs with Phase 14.
- ☐ **WASM for minimax / AlphaZero.** Compile the engines to WASM behind a feature flag; benchmark vs the JS engines. Expected 10–100× faster move computation. Critical for Connect4 (tied to `doc/Connect4_Ship_Checklist.md` Spike A on TF.js).
- ☐ **OffscreenCanvas for game render.** When games like Pong land, keep the render loop off the main thread.
- ☐ **Move ML training to a backend worker.** Tracked in `doc/Connect4_Ship_Checklist.md` Spike B — perf is one input; architectural decision is broader (mobile training, headless cluster runs, training on devices that can't keep a tab open).
- ☐ **`requestIdleCallback`** for journey events, telemetry, RUM flushes — never compete with user input on the main thread.

Phase 20 — Cache & app shell

Today: warm visits read from `localStorage` for some stores, but the HTML / JS / CSS still hits the network every time.

- ☐ **Service Worker app shell.** Cache the index HTML + the vendor chunks; serve from cache on second visit; revalidate in background. Repeat-visit Ready ≈ 0ms. Highest-leverage perceived-perf change not yet in v2.
- ☐ **IndexedDB for tournament/leaderboard JSON.** Faster than localStorage at size; survives reload. Stale-while-revalidate directly off IDB.
- ☐ **Edge response cache** on Fly for public reads (`/leaderboard`, `/bots`, `/tournaments`). Backend never sees cached requests.
- ☐ **HTTP `immutable` cache headers** on hashed asset URLs (`max-age=31536000, immutable`). Confirm Fly is doing this.
- ☐ **Persist Zustand stores** so warm boot has full state — guide progress, sound prefs, ui sort orders, etc.

Phase 21 — Architecture experiments (longshots)

Each is a meaningful re-architecture; pursue only if Tier 0 + Tier 1 data shows the gap they'd close. Listed for completeness.

- ☐ **Server-side render the landing page (/) only.** It's mostly static. React 19 RSC or a tiny SSR shell could deliver Ready ≈ TTFB. Other routes stay SPA.
- ☐ **Edge functions for `/leaderboard` / `/bots`.** Fly Functions could serve these from cache without touching the Node process, cutting cold-start exposure entirely.
- ☐ **Replace JSON-over-HTTP with gRPC-Web** for hot endpoints — smaller payloads, schema validation, generated clients.
- ☐ **Migrate hot read endpoints off Express to Hono on the edge** — lower per-request overhead than full Express middleware stack.
- ☐ **React Server Components for static-ish pages** — Settings, Puzzles, parts of Profile. Cut their JS to ~0 KB.

Section D — Instrumentation as a first-class deliverable

Performance work without instrumentation is fishing in the dark. v1 measured synthetically once and stopped. v2 makes measurement continuous.

D0 — Pre-Tier-0 instrumentation (landed 2026-05-04, `dev`)

Before committing to Tier 0 work, the toolbox is filled in so each tackled item is *measurable* before and after. Synthetic harnesses that exist today on `dev`:

Aspect	Script	What it answers
Cold-page totals	`perf/perf-v2.js`	TTFB / FCP / LCP / Ready per route × device, with `--extended-resources` for late-loading bytes
Interaction (INP)	`perf/perf-inp.js`	p50/p95 INP per common interaction × route
Backend hot endpoints	`perf/perf-backend-p95.js`	p50/p95/p99 for `/api/version`, `/api/v1/bots`, `/api/v1/leaderboard`, `/api/auth/get-session`, `/api/tournaments`
SSE round-trip (F4)	`perf/perf-sse-rtt.js`	Splits POST move → SSE event into server.lookup, server.apply, redis publish→pickup, broker pickup→write, network legs
Cold-page waterfall (F2)	`perf/perf-waterfall.js`	When the route's primary `/api/*` fires + how long it takes — sizes the prize for `<link rel=preload>` and inline initial payload
Long-task profile	`perf/perf-longtasks.js`	count / sum / max long tasks per cold load (Moto-G4 + 4× CPU throttle for mobile) — sizes Phase 1 / 1b TBT prize
Bundle composition	`perf/perf-bundle.js`	Per-chunk raw / gzip / brotli for every dist/asset; checked-in baseline for diff after Phase 1

Backend-side instrumentation: - Global `Server-Timing: handler;dur=X` middleware on every backend response (preserves route-level timings like the move POST's `lookup, apply`). CORS exposes `Server-Timing`. The waterfall + any future ad-hoc test can read server vs network breakdown for every `/api/*` call without per-handler instrumentation. - Move POST emits `lookup;dur=X, apply;dur=Y`. - SSE broker injects `_t: { publishToPickupMs, pickupToWriteMs }` into every payload (publishMs comes free from the Redis stream id).

Key bundle finding: `main.supported.js` is **1506 KB raw / 319 KB brotli** — half of total brotli bytes shipped. This is the F5 / Phase 1 target; the rest of the JS budget (16 chunks) is mostly small named splits (TrainTab, ExplainabilityTab, game, etc.).

D1 — RUM (Real-User Monitoring) (landed 2026-05-04, `dev`)

Synthetic baselines see what one Playwright run on a controlled connection sees; RUM sees what real users on real networks actually experience.

- **Client:** `landing/src/lib/rum.js` registers `onFCP / onLCP / onINP / onCLS / onTTFB` from `web-vitals`. Each metric appends to a per-tab queue; on `pagehide` (or first `visibility:hidden`) the queue is drained as a single beacon to `/api/v1/perf/vitals` via

navigator.sendBeacon with explicit application/json content type (the default text/plain is silently dropped by Express's body parser — easy footgun, captured here so we don't repeat it). Sampling: VITE_RUM_SAMPLE_RATE (default 1.0); decision is sticky for the tab.

- **Backend:** POST /api/v1/perf/vitals (route backend/src/routes/perfVitals.js). No auth, no SSE-session required. Validates each entry (allowed name set, finite numeric range 0–1e7, capped 32 vitals per beacon), persists via Prisma createMany, always returns 204. 8 vitest cases cover happy path, unknown names, garbage values, oversize beacons, DB failure.
- **Schema:** PerfVital table — anonymous (sessionId is a per-tab random hex, never a user id), no IP. Indexes on (name, route, createdAt) and (env, name, createdAt) so the obvious aggregations (p50/p75/p95 per route × device × env over time) stay cheap.
- **Smoke:** Playwright headless run hits /, fires visibilitychange → hidden, beacon arrives 204, three rows land in DB (TTFB, FCP, CLS — LCP/INP need real interaction to register).

Privacy boundary: never sends userId, never stores IP, no fingerprinting. The only durable identifier is the tab-scoped sessionId, which has no cross-tab persistence.

Next: ship to staging + prod with the next /stage cycle. After ~1 week of real-user data — or immediately, with the synthetic driver in D2 — we have an honest p50/p75/p95 baseline to diff every Tier 0 deploy against. Admin dashboard for it landed 2026-05-04 (see D3).

D2 — Synthetic RUM driver *(planned)*

D1 only fills perf_vitals when real humans load pages. Staging has near-zero organic traffic; prod takes hours-to-days to accumulate stable percentiles. So before we can compare a Tier 0 change against a "clean" baseline we need a way to *guarantee* a complete sample set. The synthetic driver is that guarantee — Playwright-scripted sessions that exercise every route on both desktop and mobile profiles and let the same web-vitals listeners + beacon do the reporting.

Script: perf/perf-rum-driver.js (TBD).

- Launches N parallel chromium contexts (default 5; configurable).
- Iterates the standard route inventory (/, /play, /tournaments, /rankings, /gym, /profile, …) per device profile (desktop, mobile w/ 4× CPU throttle via CDP, optionally 4G / 3G network throttle).
- Per route: load → wait for LCP candidate → perform a representative interaction so INP fires (button click that doesn't navigate) → trigger visibilitychange:hidden so the beacon flushes → close context. Passive page loads alone don't yield LCP / INP / CLS, so the interaction step is non-optional.
- Tags every session with a recognizable UA suffix (XO-Synthetic/1.0) so the aggregation endpoint can split organic vs synthetic via userAgent ILIKE '%XO-Synthetic/%'. No schema change, no separate ingest path — same POST /api/v1/perf/vitals.
- Backend GET /admin/health/perf/vitals gains a ?source=organic|synthetic|all filter (default organic) so the admin dashboard can show either view without polluting the real baseline.

Cadence:

- *On-demand* (always available): developer runs BASE_URL=https://xo-frontend-staging.fly.dev node perf/perf-rum-driver.js before and after a change to capture before/after percentiles in one session.
- *Scheduled* (CI): a GH Actions cron drives staging every ~6h so the staging RUM dashboard never shows empty bars. **Prod is not on the cron** — synthetic samples on prod would skew real-user numbers, so prod runs are manual-dispatch only, used when "what does INP look like under prod backend load?" is the actual question.

Caveats:

- Synthetic INP is bounded by Playwright's input-event latency, which is tighter than a real human. Treat synthetic INP as a *stability / regression* signal, not as an absolute number for budget compliance.
- Network conditions skew the picture if not pinned. Default to one desktop + one mobile-throttled profile per run; optionally include 3G / 4G via CDP Network.emulateNetworkConditions for the long-tail.
- Synthetic rows accumulate cost in perf_vitals. Cleanup is a scheduled prune (D2.1) — see below — not a per-run delete, so comparison runs across days remain possible.

D2.1 — Synthetic-row retention & data hygiene *(planned)*

The synthetic driver produces a continuous trickle of rows; without a prune, perf_vitals grows linearly forever (a 6h staging cron writing ~5 routes × 2 device profiles × ~5 vitals = ~50 rows/cycle = ~200 rows/day = ~75k rows/year per cron alone, before any prod synthetic or any organic). Cheap on day one, painful to fix at year three. So the prune ships with D2.

Goals.

1. Keep perf_vitals bounded so admin queries stay sub-100 ms even at year-scale.
2. Never delete data mid-experiment — a comparison run that started yesterday must still have its baseline rows tomorrow.

3. Default behaviour is safe: organic rows are untouched, synthetic rows are kept long enough to span a typical Tier 0 PR cycle.

What gets pruned (and what doesn't).

Source	Default retention	Rationale
Synthetic (`userAgent ILIKE '%XO-Synthetic/%'`)	14 days	Long enough to span a feature branch + review + post-merge diff. Tunable up.
Organic (everything else)	unlimited	Real-user rows are scarce on staging and irreplaceable on prod. Only prune if the table actually outgrows its index — measured, not pre-emptive.
Smoke / dev rows (`env = 'local'`)	7 days	Kept short — they're noise once the smoke run that wrote them has been verified.

Cadence. Nightly. Implemented as a recurring `scheduledJob` (type `perf_vitals_prune`) registered with the existing `scheduledJobs.js` dispatcher: handler runs the deletes, then enqueues the next run 24 h out. If the dispatcher misses a tick (process restart, env unavailable), startup recovery resets stuck RUNNING jobs and the next tick catches up — no lost data, no double-prune.

Implementation sketch.

- New handler module `backend/src/services/perfVitalsRetentionService.js` exporting `pruneSynthetic({ olderThan })`, `pruneSmoke({ olderThan })`, and a `runPrune()` orchestrator. Each delete uses the existing `(env, name, createdAt)` index — an `EXPLAIN` against the staging DB before merge confirms an index scan, not a seq scan.
- Bootstrap from `backend/src/index.js`: register the `perf_vitals_prune` handler, then enqueue the first run if no PENDING / RUNNING row already exists. Idempotent across restarts.
- Test coverage: unit tests for each prune function (boundary on the cutoff, no-op when nothing matches, error path doesn't tombstone the job), plus a handler-level test that asserts the next run is re-enqueued.

Pause / extend for long comparison runs. Two system_config keys (`getSystemConfig` / `setSystemConfig` already wired through the admin panel):

- `perf.retention.synthetic.days` (number, default `14`) — bump to `60` before a multi-week experiment, drop back to `14` afterwards.
- `perf.retention.paused` (bool, default `false`) — hard pause; the handler short-circuits to a no-op while still re-enqueueing itself so resuming is just flipping the flag back. Useful when a multi-day "hold everything" capture is in flight.

Both are read at the top of the handler so a flip takes effect on the next tick (≤ 24 h) without a backend restart.

Observability. Each run logs `{ prunedSynthetic, prunedSmoke, keptOrganic, durationMs, paused }` at info level. The numbers also land in the metrics snapshot so the admin Health dashboard's `/admin/health/sockets` history shows whether the table is stabilizing or still growing.

One-time scrub (only if needed). If any synthetic rows leak into the table *before* D2 ships its UA tag, they'd be indistinguishable from organic. Mitigation:

1. Ship D2 (UA tag + `?source` filter) and D2.1 (prune) **in the same PR** — this is the primary defence; there's no untagged-synthetic period.
2. Belt-and-suspenders: a one-shot admin endpoint `POST /admin/health/perf/vitals/scrub-synthetic-before` accepting a timestamp, runnable manually if step 1 ever fails. Behind `requireAdmin`, idempotent, returns the row count it deleted. Add only if step 1's pre-merge testing reveals a gap; otherwise skip.

Future: dedicated `source` column. The UA-suffix filter is fine at year-one scale (`userAgent` is already indexed via the `(name, route, createdAt)` composite for the hot path; the source filter only matters on infrequent admin queries). If the synthetic volume ever rises to where the `ILIKE` becomes hot, add a `source enum('organic', 'synthetic', 'smoke')` column and a backfill — but not before, since the column adds write-side cost on every beacon and the current shape is already costly enough at hundreds of rows/sec organic.

Sequencing — when does cleanup actually happen? Three distinct moments to keep separate:

1. *Before the next `/promote`* (i.e., right now): **no cleanup needed.** D1 just landed on dev; prod's `perf_vitals` is empty. The `/stage` + `/promote` we're queueing pushes code, not data.
2. *Between `/promote` and the synthetic driver going live*: **also no cleanup needed.** Real users start writing organic rows immediately; that's exactly what we want. The perf-* scripts re-baseline produces JSONs in `perf/baselines/`, never touching `perf_vitals`.
3. *Once D2 + D2.1 land*: cleanup becomes ongoing — the prune does its job nightly, queries default to `?source=organic`, the dashboard stays clean. **No "cleanup phase" between push and data collection** — the cleanliness primitives ship inside D2 itself, not as a separate gate.

D3 — Admin Health dashboard panels (landed 2026-05-04, `dev`)

The on-call surface for everything D0–D2 produces, all under `/admin/health`:

- **Real-User Web Vitals** — `GET /api/v1/admin/health/perf/vitals` — per-(route, metric) p50/p75/p95 with rating-mix bars, 1h/24h/7d toggle, env breakdown. Color-coded against the web-vitals thresholds (LCP ≤ 2.5s good / > 4s poor, INP ≤ 200ms / > 500ms, CLS ≤ 0.1 / > 0.25, …).
- **Perf Baselines (dev-only)** — `GET /api/v1/admin/perf/baselines{,/:filename}` — read-only browser of `perf/baselines/*.json` with kind filter and click-to-view JSON. Disabled on Fly.io (no `PERF_BASELINES_DIR`) because the JSONs only exist on the dev machine that ran the perf scripts. Strict filename regex + resolved-path check guard against traversal.

Tests: 6 + 8 vitest cases on the backend; UI is straightforward presentation.

Phase 17 — Per-PR perf gates

- ☐ CI runs `perf/perf.js` against a preview env on every PR; comments the diff vs `main`.
- ☐ Bundle-size guard fails PR if any chunk grows > 5% without an explicit `perf-ok` label.
- ☐ Backend test suite asserts p95 < budget for the 5 hottest endpoints (run against an in-memory load generator).

Phase 18 — Production perf dashboard

- ☐ Public-internal Grafana page: per-route Ready p75/p95 (RUM), per-endpoint backend p95, SSE channel health, error budgets.
- ☐ Weekly "perf review" cadence — owner reviews against budgets, files issues for any breach > 7 days.

Section E — Aggressive bets (re-review 2026-05-04)

A second pass over the plan exposed several large-leverage ideas that weren't in any phase. Some are nearly-free additions to existing phases; others are real moonshots. They're grouped by what they buy you, not by tier — surface them where they fit, then promote the ones the data backs after Tier 0 lands.

E1 — Free wins (add to Tier 0 / quick-wins)

These cost ≤ a day each, ship behind feature flags if needed, and should be done alongside the bundle work — most don't even require a benchmark to justify.

- ☐ **103 Early Hints from Fly.** Send `<link rel="preload">` for `vendor-react` + `main` + critical CSS *before* the backend has finished computing the HTML response. Saves 50–150ms by overlapping TTFB with preload. Fly's edge supports this; verify and wire it up.
- ☐ **Speculation Rules API** (`<script type="speculationrules">`). Chrome / Edge will prerender same-origin links the user is likely to click — no JS, no library. Add a JSON block with the home, `/play`, `/leaderboard`, and `/profile` URLs. Free instant navigation on Chromium.
- ☐ **HTTP/3 0-RTT for returning visitors.** Cuts the TLS handshake to zero on warm connections. Fly supports HTTP/3; confirm 0-RTT isn't disabled for safety reasons we don't actually need.
- ☐ **Resource hints for auth providers** — `<link rel="preconnect">` / `dns-prefetch` for `accounts.google.com` and `appleid.apple.com` in `<head>` so the OAuth round-trip starts warm. Sign-in feels ~200ms faster on cold visitors who click immediately.
- ☐ **Brotli quality 11 at build** (vs Vite's default 6). Slower build, zero runtime cost, ~5–10% smaller JS payloads. Set `compress: { algorithm: 'brotliCompress', compressionOptions: { level: 11 } }` in the Vite config.
- ☐ **CSS `image-set()`** for the hero — `background-image: image-set("hero.avif" 1x type("image/avif"), "hero.webp" 1x)`. Browser picks best format automatically; supersedes manual `<picture>` wrapping for CSS backgrounds.
- ☐ **CSS `aspect-ratio`** on every `<img>` and lazy section. Locks layout before bytes arrive — kills CLS on the home, leaderboard, and tournament pages without manual `width` / `height`.
- ☐ **BlurHash placeholders** for the hero, bot avatars, and any future user uploads. ~30 bytes per image, decoded synchronously to a 32×32 canvas — instant visual content under the real image.
- ☐ **Variable fonts.** Replace separate Inter regular/semibold/bold static files with one variable font (`Inter.var.woff2`). Roughly half the font weight on the wire.
- ☐ **`font-display: optional`** for non-critical text (e.g., Inter Tight on hero headlines). Never blocks render, even on slow networks.
- ☐ **Modern-only build by default.** Drop the legacy bundle in production (Vite still builds it). 99% of traffic is evergreen Chromium / Safari / Firefox; the legacy chunk is dead weight.
- ☐ **`scheduler.yield()`** in any loop > 5ms (bracket layout, journey derivations, leaderboard sort). Hands control back to the browser between chunks of work — keeps INP under 100ms even on Moto G4.

- ☐ **`isInputPending()`** as the cheap version when `scheduler.yield` isn't shipped: pause heavy work mid-loop if the user is touching the screen.

E2 — React re-render lockdown

The existing phases mention `useDeferredValue` and `React.memo` on `BracketMatch`, but the platform has a deeper render-cost story.

- ☐ **Atomize Zustand stores.** The current game store, journey store, and ui store each have many subscribers. Splitting each into per-domain slices (e.g., `tableStore`, `seatsStore`, `boardStore`) means an SSE state event re-renders only what actually changed. Easy diff pattern; large measurable win on TournamentDetail under live updates.
- ☐ **`useTransition` on every nav click.** Keeps the current page interactive while the next route's chunks load — visible Ready stays low even when the next page is heavy.
- ☐ **Selective hydration where applicable.** React 18 already supports it; needs explicit `<Suspense>` boundaries to opt in. Pair with Phase 1's per-route splitting.
- ☐ **Why Did You Render audit** on TournamentDetailPage, GymPage, RankingsPage. Run in dev for a session, fix every flagged cascade. One-time effort, perpetual savings.
- ☐ **React 19 Actions + `useFormState`** for sign-in, register, bot-create, skill-add. Cleaner optimistic UI and built-in pending state — replaces hand-rolled patterns.
- ☐ **React Forget (compiler memoization)** — track stability; adopt when the React team marks it production-ready. Removes most `useMemo` / `useCallback` boilerplate, often catching cases hand-written memoization missed.

E3 — Compute moonshots (Phase 14 / 19 supercharger)

The existing plan moves ML to a Web Worker. The real wins are bigger.

- ☐ **WebGPU backend for TF.js.** Newer than the WebGL backend, ~2–5× faster on supported hardware. Falls back to WebGL automatically. One config change in TF.js setup.
- ☐ **WASM SIMD for minimax.** Compile the minimax engine with the `simd` flag — 4–8× speedup over scalar WASM, which itself is ~10× faster than the JS version. Bot move latency goes from ~30ms to ~1ms; matters more for Connect4 + AlphaZero.
- ☐ **Quantize ML models to int8.** TensorFlow.js supports int8 quantization at the conversion step. 4× smaller weights on disk + on the wire, 2–4× faster inference on CPU. Trade is ~1–2% accuracy, which is well within ELO noise for our game.
- ☐ **Cache compiled WASM in IndexedDB.** WebAssembly.Module is structured-cloneable. Compile once, store, retrieve on next boot — saves the JIT pass on every reload.
- ☐ **Background-train ML bots while idle.** When the user is reading the home page or watching a demo, run a few training epochs in a Web Worker. Their bot quietly improves between sessions.
- ☐ **Speculative bot move computation.** During the user's turn, precompute the bot's response to the top 3–5 likely user moves. User makes their move → bot's response is already cached → animate immediately.

E4 — Network moonshots

- ☐ **PgBouncer in front of Fly Postgres.** Connection pooler that handles transient connection storms (deploy-time, cold-start flap). Cuts p95 tail latency on DB-bound endpoints. Standard Postgres infra; runs as a sidecar machine.
- ☐ **DataLoader for N+1 batching.** Especially on tournament-detail eager loads — fan out N participant lookups → one batched `SELECT … WHERE id IN (…)`. Battle-tested Node pattern; drop-in.
- ☐ **WebTransport (HTTP/3 datagrams) replacing SSE.** Lower latency than SSE, better connection migration when mobile users switch networks, supports unreliable channels for fire-and-forget. Big lift; do only after RUM data shows SSE is the bottleneck.
- ☐ **WebRTC peer-to-peer for PvP** once tables are seated. Server only relays signaling; gameplay traffic is browser-to-browser. Move latency drops to one round-trip on the same network. Big lift; meaningful only for PvP, not PvB.
- ☐ **Edge KV for read-heavy public reads.** Built-in bots, system config, journey config — load once at the edge, serve from memory; backend never sees these requests. Kills the request class entirely. Fly supports this via a small KV sidecar or Cloudflare Workers KV.
- ☐ **Stale-while-revalidate everywhere safe.** `Cache-Control: max-age=60, stale-while-revalidate=600` on `/leaderboard`, `/bots`, `/tournaments` list. Browser shows cached instantly, fetches fresh in background. One-line backend change.
- ☐ **Compression dictionaries.** Brotli supports shared dictionaries across requests — train a dictionary on common JSON shapes (tournaments, bots, leaderboard) and ship it. 30–50% smaller on the wire after the first request. Browser support arriving 2026.

E5 — Cache & shell expansion (Phase 20 supercharger)

The existing Phase 20 mentions Service Worker app shell. Push it harder.

- ☐ **Service Worker as the canonical realtime substrate.** SW holds one shared SSE connection for all open tabs of the app — instead of N tabs × 1 SSE = N connections, it's 1 connection multiplexed via `BroadcastChannel`. Fly per-IP connection limits stop biting.

- ☐ **Background Sync for offline moves.** User makes a move on a flaky network → SW queues it → replays when connection returns. Player never sees a failed move.
- ☐ **IndexedDB for SSE event replay.** SW persists every state event to IDB; on tab reopen / reconnect, replay all missed events locally before the live SSE catches up. Reload feels instant.
- ☐ **Periodic Background Sync** for tournament wakeups. SW pings `/api/tournaments/upcoming` every 15 minutes when offline; pushes a notification when one starts. PWA primitive, free.
- ☐ **Push notifications for cup events.** Already partially wired via `pushSubscribe.js` — extend to "your match is starting" and "your bot won" pings.
- ☐ **WebShare API** on results pages so users share their cup wins to native iOS/Android share sheets. Not perf-direct, but cuts the friction loop that drives engagement → repeat sessions where the SW shell pays off.

E6 — Predictive perf

The platform has rich session data; use it to make navigation feel psychic.

- ☐ **Predictive prefetch from user history.** Track per-user navigation transitions (e.g., "users who view profile next view leaderboard 80% of time"). Prefetch the top-1 next route on every page load. Costs one tiny model + a few KB of state.
- ☐ **Pre-render the *expected* next route** in a hidden subtree. On `/play` cup match end, the next click is almost always "Continue" or "Rematch" — both routes can be fully mounted and ready when the click happens.
- ☐ **Speculative API calls based on cursor proximity.** As the cursor approaches a tournament card, fire `/api/tournaments/:id` before the click. Hover-prefetch as a behavior model, not just a CSS pseudo-class.

E7 — Architecture longshots (Tier 4)

Real re-architectures. Each is a project. Listed for completeness so the option is visible.

- ☐ **Bun runtime for backend + tournament services.** Cold-start drops from ~500ms (Node) to ~50ms (Bun) and request throughput goes up ~30%. Bun is Express-compatible; the migration is mostly testing. Collapses Phase 6 (cold-start) entirely. Risk: Prisma + Bun stability, Better Auth compatibility.
- ☐ **Hono + edge runtime for read-only public APIs.** Move `/leaderboard`, `/bots`, `/tournaments` list to Hono on Fly's edge. ~10× faster per request than Express, runs in every region. Already in Phase 21 — promote if Bun isn't ready.
- ☐ **Cloudflare Durable Objects (or Fly Replay) for tablePresence.** Move the in-memory presence cache to a regional state primitive. Frees backend machines to scale horizontally; eliminates a hot shared-memory table.
- ☐ **gRPC-Web for hot endpoints.** Schema-validated, smaller payloads, generated clients. Already in Phase 21; revisit after Bun.
- ☐ **Replace Better Auth with self-rolled JWT + refresh.** If the cookie-size + preflight overhead in Phase 4 measurements turns out to be load-bearing, the entire auth library may be the cheapest thing to swap. High-risk; only with measurement.

Section F — Pre-launch review gaps (2026-05-04)

A second pass before starting Tier 0 surfaced gaps the existing phases don't cover, plus DB / REST patterns the user explicitly asked about that the plan was silent on.

F1 — DB join shape + preloading (Phase 2 supplement)

Phase 2 calls for an index pass + `pg_stat_statements`, but doesn't audit the *join shape* of the heaviest endpoints. Hot offenders:

- **`GET /api/tournaments/:id`** — three-level `include`:

  ```
  include: {
    participants: { include: { user: { select: { …8 cols } } } },
    rounds:       { include: { matches: true } },
  }
  ```

  Prisma issues this as ~3 SQL queries with deep joins. On a 32-player bracket: 1 tournament + 32 participants joined to users + 5 rounds + 31 matches = a single response Of ~70 rows. Watch for two issues:

  (a) `matches: true` returns *all* TournamentMatch columns including large unused fields (e.g. `metadata` JSONB, `botMoveLog` if present);
  (b) the participant→user join can pull users that are no longer competitively active. *Action:* explicit `select` on every nested include, drop unused columns; benchmark before/after.

- **`listBots()` ELO lookup uses an OR over `(userId, gameId)` pairs.**

```
where: { OR: eloPairs }   // N pairs → N OR branches
```

Postgres can plan this poorly past ~8 branches. Rewrite as a single `WHERE (user_id, game_id) IN (VALUES …)` (raw SQL) or a UNION of exact-match subqueries; or add a composite `(user_id, game_id)` index if not already present. Cheap, measurable.

- **N+1 in spectate / live-state paths.** The bot-game runner emits state events that include only ids; the client backfills with separate fetches per id. Audit `useEventStream` consumers for any pattern that triggers a fetch loop on each event arrival.
- **Audit `select` discipline platform-wide.** Prisma defaults to selecting all scalar fields when `select` is omitted on a nested include. Three places in `listBots` already do this right; verify every `findMany`/`findUnique` in `backend/src/services/*` and `tournament/src/routes/*` follows suit.

F2 — REST response preloading (NEW)

The cold-page Ready cost is dominated by waterfalls: HTML parses → JS parses → bundle queries an API → API returns → render. The plan covers the JS parse but not the *API queries during parse*.

Instrumentation landed (2026-05-04, `dev`): `perf/perf-waterfall.js` measures, per route × device, when the primary `/api/*` call fires (`requestStart` from navStart) and when its response lands (`responseEnd`). Local sanity-check (n=3, desktop, no network throttling) — every route's primary API doesn't fire until ~400ms after navStart, which is pure JS parse + React mount + `useEffect` cost (the API itself is 2–5ms locally):

Route	apiStart	apiTotal	LCP	preload prize (~)
`/`	418ms	5ms	696ms	~400ms
`/tournaments`	377ms	2ms	388ms	~370ms
`/rankings`	385ms	5ms	396ms	~380ms

LCP on `/tournaments` and `/rankings` lands ~11ms after the API returns — preload would shift the entire chain ~370ms earlier. Staging/prod prize is presumably larger (longer bundle download + bigger API total). Re-run with `--target=staging` to confirm before committing implementation effort.

Aside: the waterfall also surfaces ~7 redundant `/api/session` hits per cold page load. Filed as a separate dedupe issue; not part of F2 itself.

- ☐ **`<link rel="preload" as="fetch">`** for the top-1 API call per route, emitted in the HTML `<head>` so the request fires in parallel with bundle download. Routes + targets:
  - `/` → `/api/v1/bots?gameId=xo` (the community bot fetch)
  - `/tournaments` → `/api/tournaments`
  - `/rankings` → `/api/v1/leaderboard?period=all&mode=all&includeBots=false`
  - `/profile` (auth) → `/api/v1/bots?ownerId=…`
- ☐ **103 Early Hints with `Link: rel=preload`** (Section E1) for both static chunks *and* the API preload above. Doubles the effect: preload starts before TTFB *and* parallel to JS parse.
- ☐ **Inline initial payload into `index.html`** for routes whose data shape doesn't depend on auth or query params. Static-ish candidates: built-in bot list, system config keys the client reads first paint. Cuts an entire round-trip on `/`.
- ☐ **HTTP/2 server push fallback?** Skip — browsers have removed it. 103 Early Hints is the modern equivalent and already in E1.
- ☐ **Wire `<Link to>` hover into a fetch warmup** (extending Phase 10): on hover, pre-fire the route's primary API call and cache the response for the upcoming nav.

F3 — Connection pool, transactions, query plans (NEW)

Phase 2 mentions PgBouncer; doesn't audit what's already in place.

- ☐ **Prisma connection pool size.** Default is `num_cpus * 2 + 1`, which on Fly's `shared-cpu-1x` resolves to 3. Under modest concurrency (5+ simultaneous SSE move POSTs) this saturates and queues. Bump explicitly via `connection_limit` query param; benchmark p95 under load before/after.
- ☐ **Transaction footprint audit.** Hot multi-step transactions: bot-game runner per-move, tournament round advancement, training-session completion. Long transactions hold row locks and block concurrent reads. Profile + shrink scope where possible.
- ☐ **Prepared statement reuse.** Prisma's query engine prepares and caches statements per connection — confirm the cache is sized for the actual query variety (default 100; if exceeded, Prisma re-prepares on every call).
- ☐ **`EXPLAIN ANALYZE` the top 5 slow queries** identified in the Phase 2 `pg_stat_statements` pass. A confirmed-slow query plan is what justifies an index, not a heuristic.
- ☐ **`SELECT FOR UPDATE` audit.** `tournamentService` advancement uses row locks; verify the lock window is bounded (no awaiting external service calls inside a `$transaction`).

F4 — Phase 5 (SSE round-trip) decomposition (concrete steps)

The 0.4 data showed POST move → SSE event = 560ms / 670ms p50, with ~400ms unaccounted for between POST ack and event arrival. The single- stream `perf-sse-rtt` decomposition (2026-05-05) put 383ms p50 of that on `publishToPickup` (Redis XADD → broker XREAD wake). The follow-up concurrent load test (`perf/perf-sse-load.js`, 2026-05-05) sharply rewrote what that number represents:

- **It is not Redis throughput** — `pickupToWrite` is < 1ms p50 at every load level, and `apply` (which includes the XADD) is < 20ms p50 even at c=50.
- **It is not load-related** — at c=50 concurrent users the publishToPickup p50 is **21ms** (vs 383ms quiet), p95 106ms.
- **It is the broker's process wake-up cost on a quiet service.** The broker uses `XREAD BLOCK 30_000` (`backend/src/lib/sseBroker.js:43`). When the backend is idle, the Node process pages out / scheduler deprioritises it / Redis TCP connection's recv-buffer is empty. The first XADD that follows pays a measurable wake-up tax (~150–500ms) before `Date.now()` is sampled in `dispatchEntry`. Once the loop is hot — i.e. any time the broker has dispatched an event in the last few hundred ms — that tax is ~20ms.
- **By move-index** (staging, c=5, n=5 each):
  - move 0 (cold table, may also catch broker idle): **382ms p50, 740ms p95**
  - move 1 (warm channel): 35ms p50, 49ms p95
  - move 2 (warm channel): 55ms p50, 69ms p95

Concurrency sweep — staging, 2026-05-05 `perf/perf-sse-load.js --target=staging --sweep=1,5,10,25` then a follow-up `--concurrency=50`. 3 moves per virtual user, 80ms ramp.

c	n	apply p50/p95	publishToPickup p50/p95	pickupToWrite p50/p95	movePostAck p50/p95	botMoveTotal p50/p95
1	3	7 / 8 ms	55 / 196 ms	0 / 1 ms	166 / 197 ms	257 / 367 ms
5	15	7 / 8 ms	55 / **740 ms**	0 / 0 ms	189 / 260 ms	324 / 932 ms
10	30	7 / 18 ms	32 / 406 ms	0 / 0 ms	195 / 304 ms	278 / 612 ms
25	75	15 / 45 ms	25 / 406 ms	0 / 0 ms	291 / 556 ms	391 / 721 ms
50	150	19 / 41 ms	21 / 80 ms	0 / 0 ms	396 / 604 ms	453 / 633 ms

Reading the table:

- **`publishToPickup` p50 *drops* with load**, from 55 → 21ms. p95 spikes at c=5/10 (740 / 406ms) but is dominated by move-0 cold-wake (see breakdown above). At c=50 the p95 is back to 80ms because the broker loop never goes idle long enough to catch a cold wake-up.
- **`apply` p95 climbs cleanly** (8 → 45 ms = 5.6× at c=25, then 41ms at c=50 — Postgres holds up). This is the only metric where load amplification is monotone and clean.
- **`pickupToWrite` is flat at 0** at every level — broker dispatch loop is not a bottleneck, which kills one Phase 5 sub-task ("audit redis pub/sub round-trip" — answer: it's fine).
- **`movePostAck` p50 ~doubles** by c=50 (166 → 396 ms), driven by apply growth + Node event-loop pressure. This is the dominant user-perceptible regression under load.
- **No fatal failures** at c=50 (50/50 users completed all 3 moves).

This rewrites Phase 5's ROI math and step list. The original in-process-fanout / pubsub-shortcut steps below are still valid, but the headline win is no longer "~300 ms saved on every move": it's "~300 ms saved on the *first* move per fresh table, *only* on quiet services."

- ☒ **Time-trace one move end to end.** *(spike landed 2026-05-04 on dev)* — `backend/src/routes/realtime.js` POST `/rt/tables/:slug/move` emits `Server-Timing: lookup;dur=X, apply;dur=Y` (lookup = caller resolution + table findFirst; apply = applyMove + DB writes + Redis XADD). `backend/src/lib/sseBroker.js#dispatchEntry` injects `_t: { publishToPickupMs, pickupToWriteMs }` into the JSON payload of every SSE frame (publishMs derived from the Redis stream id `<ms>-<seq>`). `perf/perf-sse-rtt.js` parses both and reports six decomposition rows: server.lookup, server.apply, network (POST RTT − server), redis publish→pickup, broker pickup→write, network (ack→event − server). **Local sanity-check (n=10):** every leg ≤2ms; total matches client-measured `playerEventMs`. Staging + prod numbers TBD after promote — that's where the ~400ms lives.
- ☐ **Keep the broker hot — kill the cold-wake penalty.** *(NEW, top priority based on 2026-05-05 load evidence.)* The cheapest possible fix: add a 1Hz keepalive XADD into `events:tier2:stream` (no-op payload, suppressed at dispatch). Cost: 86k extra stream entries/day, negligible Redis CPU. Benefit: `publishToPickupMs` p50 drops from ~383ms to ~20ms on every quiet-traffic environment (prod nights, staging, dev). Or reduce `XREAD_BLOCK_MS` from 30_000 to 1_000 so the loop iterates every second — same effect via a different lever; pick whichever has cleaner failure modes. Validate with `perf/perf-sse-load.js --concurrency=1` before/after.
- ☐ **Confirm SSE response is unbuffered.** Express + compression middleware can buffer SSE frames if not disabled per-route. Verify

`Content-Encoding: identity` (or no-op compress) on `/api/v1/events/stream` — gzip would buffer the entire stream.

- ☐ **Disable Nagle's algorithm** on the SSE response socket (`socket.setNoDelay(true)`). On Node + Express this is rarely automatic; can shave 40ms per write.
- ☐ **Audit Redis pub/sub round-trip.** SSE channels go through Redis (per `useEventStream`). Time the publish→deliver gap on the same VM. If it's > 50ms, the pubsub topology needs work (sharding, in-process fanout for same-instance subscribers).
- ☐ **Same-instance shortcut.** If the publishing process and the SSE-serving process are the same Fly machine (true for our single-machine setup), skip Redis entirely and dispatch via in-memory event emitter. Redis only kicks in when machines need to fan out to subscribers on other VMs.
- ☐ **Coalesce duplicate events.** A single move can emit `state(moved)` + `lifecycle(...)` + journey progress; if these fire serially with their own SSE writes, latency multiplies. Pack into one frame where possible.
- ☐ Re-measure with `perf-sse-rtt.js`. Target: POST move → SSE event ≤ 200ms p50 staging, ≤ 250ms p50 prod.

F5 — Phase 1 specifics from the visualizer (concrete byte targets)

Phase 1 gives directional sub-steps but no per-chunk byte budget. After running `VISUALIZE=1 npx vite build` once and reading the output, set hard targets for each chunk and fail CI on regression:

- ☐ `vendor-react` ≤ 50 KB gz (current ~82 KB — Preact compat eval)
- ☐ `vendor-charts` (`recharts`) ≤ 25 KB gz (post-uplot swap)
- ☐ `vendor-auth` (Better Auth + better-fetch + zod) ≤ 30 KB gz
- ☐ `main` ≤ 150 KB gz (current 411 KB — biggest cut)
- ☐ Each lazy route chunk ≤ 60 KB gz
- ☐ `game-xo` ≤ 30 KB gz
- ☐ Total first-paint JS ≤ 250 KB gz on Home (current 490 KB)

If any chunk exceeds budget, the PR fails CI without a `perf-ok` label. Re-bake budgets after Phase 1 lands.

F6 — Optimistic move rendering (perceived-perf, NEW)

The 560–670ms POST → SSE round-trip is the ceiling on Phase 5; below that ceiling the user still feels every move's latency unless the client compensates. The existing Phase 12 ("animation budgets") does not address this.

- ☐ **Render the player's move locally the instant the click lands.** Update the local board state, animate the mark, play sound — all before the POST returns. Reconcile when the SSE `state(moved)` event arrives.
- ☐ **Predict the bot's response.** For built-in minimax bots, the same minimax engine ships in the client bundle. After the player's move, run minimax locally to predict the bot's response, render it as a "ghost" preview after ~150ms, then reconcile with the real SSE event. If predictions match (they will most of the time for minimax), latency feels zero.
- ☐ **Disable click during reconciliation window.** Prevents double-tap from creating an inconsistent local state.
- ☐ **Visual cue if reconciliation rolls back.** A subtle shake + color flash when the local prediction was wrong (rare for built-in bots; meaningful for ML bots).

F7 — Streaming + progressive rendering (perceived-perf, NEW)

The plan mentions "streaming responses for tournament detail" inside Phase 9, but doesn't lay out the priority order or the rendering strategy. Expand:

- ☐ **`TournamentDetailPage` priority cascade.** Render the page header (name, status, time) immediately from URL params or the list-cache. Then `<Suspense>` boundaries in this order: bracket → participant table → match history → coaching cards. Each `<Suspense>` resolves independently so the slowest segment doesn't block faster ones.
- ☐ **NDJSON / chunked transfer for large lists.** `GET /api/tournaments/:id` returns a single JSON body today. Switch to NDJSON: header chunk, then participant chunks, then match chunks. Parse and render incrementally on the client.
- ☐ **Stale-while-revalidate as the default for lists.** When the user navigates back to `/tournaments`, show the cached list *instantly* (we already had it in memory), then refresh in the background. Same pattern for `/leaderboard`, `/bots`.
- ☐ **Skeleton density matches content shape.** Cards with avatar circles, table rows with proportional column widths, bracket shapes that mirror the round-N layout. Misshapen skeletons cause layout flicker on resolve and read as worse than no skeleton at all.

F8 — Open measurement gaps (2026-05-05)

After closing the authed-endpoint gap on 2026-05-05, the remaining synthetic blind spots, ranked by likely-to-find-real-issues:

#	Gap	What's missing	Why it matters	Effort
1	**Cold-authed page-level perf**	perf-v2 only does `cold-anon`. We have authed *endpoints* (p95) but not authed *pages* (Ready/FCP/LCP).	Phase 1c is scoped from endpoint math (200→143 ms p50). Actual cold-authed page Ready could be 300 ms slower than cold-anon for *all* signed-in users — we're guessing the win.	1d
2	**TournamentDetail page**	Not in `perf-v2`'s route list. The 1900-line render path Phase 13 calls out.	Heaviest single surface, most-engaged users, completely unmeasured. Can't tell if Phase 1's lazy-load helps until we benchmark.	1d
3	**Live SSE under realistic load** *(closed)*	~~`perf-sse-rtt` measures **one** move at a time, isolated.~~ Closed 2026-05-05 by `perf/perf-sse-load.js`. Findings rewrote Phase 5's scope — see §F4 above. tl;dr: fan-out load doesn't degrade `publishToPickup`; the 383 ms prod number is **cold-broker-wakeup** on the *first* move into a fresh table, and warm/under-load moves all complete in <60 ms p50.	closed	
4	**Repeat-visit / warm cache**	Only cold-anon context. SW app shell (Phase 20) promises ~0 ms repeat Ready — completely unvalidated. HTTP cache hit-rates unknown.	Repeat visits are the *majority* of sessions for engaged users. Zero data on the cache regime that matters most.	1d
5	**DB time as fraction of p95**	Endpoint p95 is wall-clock — could be 90% DB or 10% DB. Phase 0.3 (RED metrics) hasn't shipped.	Phase 2 is "deferred for lack of evidence" — but the evidence collection itself is the deferred work. Circular.	2-3d
6	**iOS Safari**	Moto G4 (Chromium emulation) is the only mobile signal.	iOS Safari has different scheduling, cache eviction, and SSE handling (6-conn limit per origin is a known footgun). Significant market segment, zero data.	1d
7	**CLS (Cumulative Layout Shift)**	LCP is tracked, CLS is not.	Hero swap, late-loading auth UI, modal reflows all *could* cause shift (budget ≤0.1 per Web Vitals). Probably fine but the not-knowing is itself the gap.	0.5d

Suggested cadence: add one gap per sprint, in rank order. Don't batch — each gap-closing PR is its own measurement → finding cycle.

My recommendation for next gap to close (after Phase 1 lands): gap **#1 (cold-authed pages)**. It directly validates Phase 1c's expected ROI, and we already have the `xo_perf` token plumbing from the `um perfuser` CLI. Gap **#3 (SSE under load)** closed 2026-05-05 — see §F4 above for the full evidence and revised Phase 5 scope.

What stays *out of scope* for the synthetic harness even now: cross-region latency (not actionable from a single test runner), tab-throttling behavior (browser-internal, hard to script), and battery / energy profiling (not a perf-budget conversation). Those go to RUM / production telemetry if they ever become a question.

Out of scope (and why)

Option	Reason skipped
SSR / Next.js	Major rewrite. Current SPA already hits FCP ~130ms; SSR's wins (LCP) are addressable cheaper via Phase 3 (image) + Phase 1 (bundle). Revisit only if mobile LCP > 700ms after Phases 1+3.
CDN / Cloudflare in front of Fly	Discussed in v1 — still ruled out for cost / ops; Fly's anycast already gets us most of the way. Revisit when international traffic is non-trivial.
Redis / external cache	Same call as v1 — in-process cache + per-service warmth is enough. SSE pubsub already uses Redis.
Drizzle / Prisma rewrite	Same call as v1 — query rewrite cost > expected gain. Phase 5 (driver) already in.
Move ML training to backend	Tracked separately in `doc/Connect4_Ship_Checklist.md` Spike B — perf is one input but architectural decision is broader.
Tensorflow.js conversion	Same — `doc/Connect4_Ship_Checklist.md` Spike A.

Sequencing *(updated 2026-05-02 from snapshot data)*

The original sequencing was a guess; the snapshot data lets us tier the phases by expected impact. **Tier 0 must land before Tier 1 even gets a benchmark slot** — otherwise the bundle floor masks any other win.

Tier 0 — sorted by combined real + perceived benefit (2026-05-05)

Ranking criteria: **(1) ms saved per request × frequency × user reach = total real impact**, **(2) user-visible delta on the click-to-feel floor = perceived impact**. Sequencing constraints noted inline where a later phase depends on an earlier one shipping first.

Rank	Phase	Real ms benefit	Perceived	Effort	Status
—	3	mobile img_kb 888→50, desktop 888→174 (−94/−80%)	low/mid	1d	□ shipped 2026-05-05
1	1	mobile FCP 1400→~700ms (−700ms × every cold visit)	**huge**	3-5d	next
2	5	move→state 577→~270ms p50 (−300ms × every move)	**huge**	3-5d	sequenced after 1
3	1d	mobile FCP/LCP −1200ms, desktop −580ms (perceived)	**huge**	1d	parallel with 1
4	17	locks Phase-1 gains; direct ms = 0	meta	0.5d	sequenced after 1

Completed (this Tier 0 round):

- □ **Phase 3** — Hero image diet. **Shipped 2026-05-05** in v1.4.0-alpha-4.1. Responsive WebP via CSS media query: 50 KB mobile / 174 KB desktop, down from 888 KB single asset. Measured: img_kb on Home Mobile dropped 888 → 50 (−94%); mobile TBT 62 → 50ms (−19%). Mobile Ready/LCP movement small because mobile is JS-parse-bound, not byte-bound — the win lands primarily on bytes-over-the-wire.

Active queue (sorted by combined real + perceived benefit):

1. **Phase 1** — Bundle audit + per-route splitting. **The single biggest remaining Tier 0 lever** — affects every cold visitor on every page. Concrete sub-steps come from the visualizer; expected to drop `main` from 411 KB gz to ≤ 200 KB gz and mobile FCP from ~1400 ms to ~700 ms (mobile Ready is a flat ~2050 ms band — JS parse dominates). Real benefit ≈ 700 ms × every cold-anon and cold-authed visit; perceived benefit huge (FCP is the most visible user metric).
2. **Phase 5** — SSE round-trip latency. *Scope narrowed after the 2026-05-04 Fly-Replay reliability fix.* Remaining: 577 ms p50 / 926 ms p95 prod for POST move → SSE state event. F4 decomposition shows the long pole is `publishToPickup` (Fly Upstash pub/sub) at 383 ms p50, *not* code we control: `server.lookup` 4 ms, `server.apply` 10 ms, `broker.pickupToWrite` 0 ms. Cutting it further means moving pub/sub closer (Redis on Fly proper, in-region replica) or collapsing the event hop entirely (write directly to the originating connection's response, skipping pub/sub for the single-machine fast path). Real benefit ≈ 300 ms × every PvP / PvB move; perceived benefit huge (this is the click-to-result feel). **Sequenced after Phase 1** — the bundle work likely shrinks the perceived gap on its own; re-measure before scoping.
3. **Phase 1d** — Instant-ready hero (perceived-only). Inline a static SVG of the demo board in `landing/index.html` so first paint shows the hero without waiting for JS. Live `DemoArena` takes over on hydrate (cross-fade or in-place swap). Real ms = 0; perceived ms = −1200ms FCP/LCP mobile / −580ms desktop — biggest perceived-perf single move available. **Stacks with Phase 1, ships in parallel** — independent code path. ~1 day. (Full spec under "Phase 1d" in Section A.)
4. **Phase 17** — CI bundle-size guard. Fail PR on > 5% chunk growth. *Sequenced after Phase 1.* Direct ms = 0; meta-value high (locks in Phase 1's gains so future feature work doesn't re-bloat them). Half-day to wire.

(**Phase 1b — PlayVsBot deep-dive** dropped from Tier 0 on 2026-05-05. The latest baseline shows PlayVsBot Ready at 991 ms desktop / 2058 ms mobile — same band as every other route. The "uniquely slow" rationale is gone. Moved to Tier 2; will be re-promoted only if post-Phase-1 measurements show PlayVsBot specifically lagging.)

Measurement gaps to close before declaring Tier 0 "done" Today we baseline cold-anon synthetic only. Before claiming the Tier 0 floor is real, two gaps need to close:

- □ **Authenticated-route p95** — *Measured 2026-05-05 staging.* Wired via `um perfuser` (CLI command that creates a synthetic test user and mints a Better Auth JWT) + `PERF_AUTH_TOKEN` in `perf-backend-p95.js`. **All 6 authed endpoints under the 200**

ms p95 budget: `/users/me/roles` 142ms, `/notifications` 119ms, `/preferences` 137ms, `/hints` 127ms, `/bots/mine` 157ms, `/guide/preferences` 135ms. Authed dispatch adds ~10ms p50 over anon — negligible.

Client orchestration audit (2026-05-05). Reviewed the actual cold-authed critical path in `landing/src/components/layout/AppLayout.`

- `landing/src/store/guideStore.js`. The fan-out is *less wasteful than feared* (only 2-3 endpoints, not 6 — `/me/roles`, `/me/preferences`, `/bots/mine` are page-specific not on landing) but still has avoidable serialization:

#	Issue	Location	Impact (p50)
1	`api.users.sync` called **twice** in two parallel effects	`AppLayout.jsx:288` and `:439`	+70ms wasted RTT
2	Effect 1 serializes sync → `guide/preferences`	`AppLayout.jsx:288→296` (`hydrate()`)	unavoidable (hydrate needs User row)
3	Effect 2 serializes sync → `users/me/notifications`	`AppLayout.jsx:439→441`	could parallelize *if* sync is dedupe'd
4	`api.users.sync` is not memoized — no in-flight promise share	`landing/src/lib/api.js:139`	ground-truth for #1 / #3

Critical path today: max(effect 1, effect 2) ≈ **200 ms p50 / 262 ms p95**.

- Effect 1: sync (70ms) → guide/prefs (64ms) = 134-205ms serial
- Effect 2: sync (70ms) → notifications (73ms) = 143-262ms serial, runs in parallel to Effect 1.

With sync deduplication (memoize the in-flight `users/sync` promise so both effects share one round-trip): critical path drops to ~143 ms p50 / ~205 ms p95 — a **~60 ms / 30% cut** on every cold-authed first paint. Single-PR fix in `landing/src/lib/api.js` (wrap `users.sync` in a per-token in-flight `Map<token,Promise>` pattern; clear the entry on settle so retries still work).

Filed as **Phase 1c — cold-authed orchestration cleanup** in Tier 1 (rank #1 there — cheapest meaningful authed-paint win in the plan).

- **DB time as a fraction of endpoint p95** — Phase 2 is currently deferred for "lack of evidence", but endpoint p95 is wall-clock and could be 90% DB or 10% DB; we can't tell. Wire OpenTelemetry or pino-trace into the endpoint hot paths (Phase 0.3) to settle the Phase 2 stay-or-promote question definitively.

Tier 1 — sorted by combined real + perceived benefit

After Tier 0, **re-run `perf-v2.js`** and re-measure. The remaining gap between the new numbers and the budgets table determines what stays. Sort below is best-current-estimate; promote / demote as the post-Phase-1 numbers come in.

Rank	Phase	Real ms benefit	Perceived	Effort
1	1c	cold-authed FCP 200→143ms p50 (−60ms × every authed paint)	mid	0.5d
2	13	TournamentDetail render path + memoization	mid	2-3d
3	7	mobile critical-CSS, transform-only animations	mid	1-2d
4	8	skeletons (pure perceived-perf — no real ms)	high	1-2d
5	aux	Better Auth rate-limit whitelist (synthetic only)	none	0.5d

1. **Phase 1c — cold-authed orchestration cleanup.** *Filed 2026-05-05 from the client orchestration audit (above).* Memoize `api.users.sync` with a per-token in-flight `Map<token,Promise>` so the two parallel effects in `AppLayout.jsx` (`:288` and `:439`) share one round-trip. Critical path drops from ~200 ms p50 / 262 ms p95 to ~143 ms p50 / 205 ms p95 — a **30% cut on every cold-authed first paint**. ~30 LOC in `landing/src/lib/api.js`. **Lands after Phase 1.**
2. **Phase 13 — Tournament page** is the heaviest single surface (1900-line render path). Phase 1 should already lazy-load it, but the page itself still needs Suspense splitting + memoization + pagination on the round/match lists. Affects only users on that route, but they're our most-engaged cohort.
3. **Phase 7 — Mobile-specific.** Mobile FCP is ~3× desktop today — parse cost on Moto G4 CPU. If Phase 1 doesn't bring it to budget on its own, critical-CSS extraction + transform-only animations are the next levers. Pairs with Phase 1; only ranks below 1c/13 because it's a "mobile remainder" — needs Phase 1 to land first to know the actual residual.

4. **Phase 8 — Skeletons everywhere new** (TournamentDetailPage, TournamentsPage, BotProfilePage, ProfilePage). Pure perceived-perf pass — the actual route isn't faster, just feels faster while chunks load. Pairs with Phase 1's `<Suspense>` boundaries (the skeleton *is* the fallback). Real-ms impact = 0.
5. **Better Auth rate-limit whitelist for synthetic.** `get-session` 90/200 ok at concurrency 5 in prod. Real users won't hit this; the synthetic harness needs an exemption (allowlist by IP or signed header) so future perf runs aren't polluted by 429s. **Measurement hygiene only — no user-visible benefit.**

(Phase 0.2 RUM dropped from this list — D1 RUM beacon and admin Web Vitals dashboards already shipped via `f6b7079` and `507f7f6`.)

Quick wins — high leverage, can run alongside any tier

These don't need to wait for Tier 0 and don't conflict with later work. Order by gut-feel impact:

- **Service Worker app shell** (Phase 20). Repeat-visit Ready ≈ 0ms. Highest-leverage perceived-perf change in the plan.
- **Speculation Rules API** (Section E1). Native Chromium prerender for same-origin links — zero JS, zero library, instant nav on every cold-anon Chromium visitor.
- **103 Early Hints** (Section E1). Preload critical chunks before TTFB. 50–150ms shaved off cold first paint, free with Fly support.
- **Critical CSS inlined into `<head>`** (Phase 7). Cheap mobile FCP win the platform never had.
- **Materialized view for the leaderboard** (Phase 2). Kills a hot DB path forever; simple Postgres feature.
- **WASM SIMD minimax** (Section E3). Bot move latency from ~30ms to ~1ms. Becomes load-bearing once Connect4 lands.
- **CSS `aspect-ratio` + BlurHash placeholders** (Section E1). Free CLS kill + perceived-instant images.
- **`content-visibility: auto`** on long off-screen sections (Phase 7, Phase 13). One CSS line per container; free render skip.
- **Brotli quality 11** (Section E1) + **Brotli at the edge** (Phase 1). Cumulative 15–25% smaller wire bytes for zero runtime cost.
- **Atomize Zustand stores** (Section E2). One pattern fix; large re-render cut on TournamentDetail under live SSE updates.
- **`scheduler.yield()` in heavy loops** (Section E1). Pulls INP under 100ms on Moto G4 with no algorithmic change.
- **Variable fonts + `font-display: optional`** (Section E1). ~50% smaller font weight + never-blocking render.
- **Pre-warm caches on boot** (Phase 2). Built-in bots, puzzles, system config. The first user stops paying the miss.
- **Resource hints for OAuth providers** (Section E1). Sign-in feels ~200ms faster on cold visitors.

Tier 2 — promote *only* when data shows them as a bottleneck

These were Tier 1 in the original draft but the snapshot doesn't support attacking them yet. Promote to Tier 1 only when re-measurement shows them on the critical path.

- **Phase 2** — DB index pass. Backend p95 confirmed healthy across five endpoints in the 2026-05-05 baseline (no endpoint p95 over 140 ms). Until RED metrics show DB time > target on a real surface, this stays speculative.
- **Phase 4** — Cross-service network shape. Needs measurement before it earns a slot.
- **Phase 6** — Backend cold start. Now in iad with `auto_stop_machines` policy in place from `29542f7`. Needs cold-start measurement to decide whether the policy is worth the cost.
- **Phase 1b** — PlayVsBot deep-dive. *Demoted from Tier 0 on 2026-05-05.* Latest baseline shows PlayVsBot Ready at 991 ms desktop / 2058 ms mobile — within the same band as every other route. Re-promote only if post-Phase-1 numbers show PlayVsBot specifically lagging.

(Phase 5 was here in the 0.4 draft. Promoted to Tier 0 — see above.)

Tier 3 — perceived-perf polish, after the floor moves

These look like wins on a fast platform but reading "Refreshing…" on a 2-second cold page just adds noise. Land after Tier 0 + Tier 1.

- Phase 9 (optimistic / streaming everywhere)
- Phase 10 (hover prefetch on tournament cards / bot rows)
- Phase 11 (route transition polish — view-transitions API)
- Phase 12 (in-game animation budgets)
- Phase 14 (Gym worker-thread inference + WASM)
- Phase 15 (Guide INP audit)
- Phase 16 (Tables-as-primitive overhead)
- Phase 18 (production perf dashboard) — wait for RUM data first.

Tier 4 — architecture experiments (Phase 21 + Section E7)

These are re-architectures, not phases. Pursue only if Tier 0 + Tier 1 + Tier 2 + Tier 3 data still shows the gap they would close.

- **Bun runtime for backend + tournament** (E7). Collapses Phase 6 cold-start entirely; ~30% throughput gain; biggest single win in this tier.

- **Hono on the edge for read paths** (Phase 21 / E7). Promote first if Bun runs into compatibility issues.
- **WebTransport replacing SSE** (E4). Lower latency + better mobile connection migration. Big lift; only after RUM data.
- **WebRTC peer-to-peer for PvP** (E4). Move latency drops to one same-network round-trip once tables are seated. PvP-only payoff.
- **Cloudflare Durable Objects for tablePresence** (E7). Frees backend to scale horizontally.
- **SSR / RSC for `/`** (Phase 21). Cheaper alternatives in Tier 0/1 may close the gap first.
- **Edge functions for hot reads** (Phase 21). Subsumed by Edge KV in E4 if that lands first.
- **gRPC-Web on hot endpoints** (Phase 21 / E7). Revisit after Bun.
- **Self-rolled JWT replacing Better Auth** (E7). Only with measurement showing the auth library is the bottleneck.
- **Preact compat alias** (longshot from Phase 1).

Re-measure cadence

- After Tier 0 lands → run `perf-v2.js --target=staging --warmup` and append a new `Performance_Snapshot_<date>.md`.
- After each Tier 1 phase → same.
- Tier 2 phases require Phase 0.3 RED metrics to be in place first.

Let the data drive every promotion. If a phase lands and the next snapshot doesn't show movement, the phase is *landed but ineffective* and the assumption gets revisited.